PageRank su Architetture Ibride

Parallelizzazione con OpenMP e CUDA

Emanuele Cecchia

Corso di Programmazione su Architetture Parallele
A.A. 2025–26 — Prof. Andrea Formisano

25 ottobre 2026

Abstract

Il progetto implementa e analizza l'algoritmo PageRank su architetture parallele, confrontando quattro versioni: una sequenziale, che fa da riferimento, una basata su OpenMP, una che usa la sola GPU e una ibrida che distribuisce il carico fra CPU e GPU. Sequenziale e OpenMP condividono lo stesso file sorgente e differiscono solo per le opzioni di compilazione, così che il riferimento non possa divergere dal codice parallelo né essere falsato dai costi di gestione dei thread. Allo stesso modo, la versione solo GPU è quella ibrida senza lavoro per la CPU: le due condividono l'eseguibile, e le distingue un'opzione a tempo di esecuzione. Il grafo è memorizzato trasposto in formato CSR, scelta che rende il nucleo di calcolo privo di scritture concorrenti e quindi parallelizzabile senza sincronizzazione. La correttezza è verificata contro tre implementazioni indipendenti. Le misure sulla versione OpenMP, condotte su nove grafi reali e due processori, mostrano uno speed-up di 2,6–3,5× su 4 core e di 6,9–11,9× su 12 e, soprattutto, una saturazione oltre il numero di core fisici che identifica il calcolo come *memory-bound*: è questo il dato che motiva quantitativamente il passaggio alla GPU.

La versione ibrida è stata misurata su cinque GPU di tre macchine. Da sola, una RTX 2080 Ti è da 1,4 a 2,9 volte più veloce dei 24 thread della CPU a 12 core. Sulla GPU, dare alle righe più lunghe un warp o un blocco di thread, invece di un thread solo, la rende fino a 19 volte più veloce sui grafi dominati da poche righe lunghissime. Dividere il lavoro fra CPU e GPU conviene quando le due, usate da sole, impiegano tempi simili: su un portatile, dove una GTX 1050 Ti e 4 core vanno quasi alla pari, la versione ibrida è fino a 1,8 volte più veloce della più rapida fra le due. Con GPU più veloci conviene quasi sempre lasciare tutto alla GPU, perché a ogni iterazione la CPU deve ricevere l'intero vettore dei contributi, e quella copia costa più del lavoro che la CPU restituisce.

Indice

1 Descrizione del problema affrontato **3**
- 1.1 Il problema . . . 3
- 1.2 Damping factor e Dangling Mass . . . 3
- 1.3 Forma matriciale . . . 4
- 1.4 Algoritmo . . . 4
- 1.5 Necessità della parallelizzazione . . . 5

2 Metodologia adottata per la soluzione **5**
- 2.1 Struttura del codice . . . 5
- 2.2 Rappresentazione dei dati: CSR . . . 6

2.3 Formato binario e pre-elaborazione 8
2.4 Precisione: `float` contro `double` 10
2.5 Validazione del file in ingresso 10
2.6 Implementazione sequenziale e OpenMP 11
2.7 Implementazione ibrida OpenMP + CUDA 12
2.8 Strategia di verifica 17

3 Risultati sperimentali **18**
3.1 Ambienti di prova 18
3.2 Istanze di prova 18
3.3 Correttezza 19
3.4 Misura dei tempi 20
3.5 Scalabilità della versione OpenMP 20
3.6 Confronto fra le varianti `float` e `double` 23
3.7 Risultati della versione ibrida 24

4 Valutazioni e osservazioni **29**
4.1 Punti di forza 29
4.2 Punti deboli e limiti 29
4.3 Possibili miglioramenti 30
4.4 Valutazione complessiva 30
4.5 Codice sorgente e guida all'uso 31

1 Descrizione del problema affrontato

1.1 Il problema

PageRank assegna a ogni nodo di un grafo orientato un punteggio di importanza, definito ricorsivamente: un nodo è importante se nodi importanti puntano ad esso. Nato per ordinare le pagine web in base alla struttura dei collegamenti ipertestuali [1], viene oggi applicato anche a reti sociali, reti di citazioni e reti biologiche.

La definizione è ricorsiva e si risolve iterativamente: si parte da una distribuzione uniforme e la si raffina finché non cambia più — in pratica, finché la variazione non scende sotto una soglia prefissata, oppure fino a un numero massimo di iterazioni. Equivale al modello del *navigatore casuale*, che segue collegamenti scelti a caso: il PageRank è la frazione di tempo che costui trascorre su ciascuna pagina. Ne discendono due proprietà della formula — il totale vale 1, perché il navigatore è sempre da qualche parte, e un collegamento pesa tanto meno quanti più ne partono dalla stessa pagina.

Formalmente, dato un grafo orientato $G = (V, E)$ con $N = |V|$,

$$\mathrm{PR}_{t+1}(v) \;=\; \frac{1-d}{N} \;+\; d\left(\sum_{u\,:\,u\to v} \frac{\mathrm{PR}_t(u)}{\mathrm{outdeg}(u)} \;+\; \frac{\mathrm{dangling}_t}{N}\right) \tag{1}$$

dove i simboli hanno il seguente significato:

$\mathrm{PR}_t(v)$	*PageRank*: punteggio del nodo v alla t-esima iterazione
$\mathrm{outdeg}(u)$	numero di archi uscenti da u
d	*damping factor*, convenzionalmente 0,85
$\mathrm{dangling}_t$	*dangling mass*: punteggio complessivo dei nodi senza archi uscenti $\mathrm{dangling}_t = \sum_{u\,:\,\mathrm{outdeg}(u)=0} \mathrm{PR}_t(u)$
N	numero di nodi

1.2 Damping factor e Dangling Mass

Il termine $(1-d)/N$: un gruppo di pagine che si citano solo fra loro assorbirebbe, iterazione dopo iterazione, tutto il punteggio del grafo, perché i collegamenti vi entrano e nessuno ne esce. Con probabilità $1-d$ il navigatore abbandona i collegamenti e salta su una pagina qualsiasi: questo impedisce l'accumulo e rende ogni nodo raggiungibile da ogni altro. È anche l'unica via d'uscita da un nodo senza archi uscenti, dove con $d = 1$ il navigatore resterebbe intrappolato per sempre.

Il termine $\mathrm{dangling}_t/N$: un nodo senza archi uscenti non ha modo di ridistribuire il proprio rank. Il salto casuale lo libera con probabilità $1-d$, ma la quota d restante cercherebbe un collegamento che non esiste: è questa che il termine raccoglie e ridistribuisce. Ignorandolo, ad ogni iterazione una frazione della massa totale sparirebbe e il vettore cesserebbe di essere una distribuzione di probabilità. Nei dataset utilizzati questo non è un caso limite: sono dangling il 15,6 % dei nodi di `web-Google`, il 44,6 % di `cit-Patents` e il 57,7 % di `web-NotreDame` (Tabella 2). La conseguenza pratica è che $\sum_v \mathrm{PR}(v) = 1$ costituisce un test di correttezza.

Con entrambi i termini l'iterazione descrive una catena di Markov sui nodi del grafo: da ogni nodo — cicli chiusi e vicoli ciechi compresi — si può sempre raggiungere qualunque altro. Questo ci garantisce un'unica distribuzione stazionaria.

1.3 Forma matriciale

La stessa iterazione si scrive in forma vettoriale. Sia $\mathbf{MT} \in \mathbb{R}^{N\times N}$ la *matrice di transizione* definita da

$$\mathrm{MT}_{vu} = \begin{cases} 1/\mathrm{outdeg}(u) & \text{se esiste l'arco } u \to v, \\ 0 & \text{altrimenti,} \end{cases} \tag{2}$$

e sia $\mathbf{1}$ il vettore di tutti uni. Detto $\mathbf{PR}_t$ il vettore dei punteggi (PR(1), ... , PR(N)), l'iterazione (1) diventa

$$\mathbf{PR}_{t+1} \;=\; \frac{1-d}{N}\,\mathbf{1} \;+\; d\left(\mathbf{MT}\,\mathbf{PR}_t + \frac{\mathrm{dangling}_t}{N}\,\mathbf{1}\right), \qquad \mathrm{dangling}_t = \mathbf{w}^{\top}\mathbf{PR}_t \tag{3}$$

dove $\mathbf{w}$ è il vettore indicatore dei nodi dangling ($w_u = 1$ se $\mathrm{outdeg}(u) = 0$, altrimenti 0).

Il prodotto $\mathbf{MT}\,\mathbf{PR}_t$ che compare nella (3) è una moltiplicazione matrice-sparsa per vettore (SpMV, Sparse Matrix-Vector Multiplication): una matrice sparsa moltiplica un vettore denso e ne produce uno nuovo, sempre denso, nella forma generica $\mathbf{y} = \mathbf{Ax}$. È questa l'operazione che il progetto parallelizza. Per non pagare gli zeri né in spazio né in tempo si memorizzano i soli elementi non nulli di $\mathbf{MT}$, usando formati come CSR (Compressed Sparse Row), che vedremo nella Sezione 2.2.

1.4 Algoritmo

Algorithm 1 PageRank per iterazione di potenza (formulazione *pull*)

```
PR[v] ← 1/N per ogni v ∈ V
repeat
    dangling ← 0
    for all u ∈ V do
        if outdeg[u] = 0 then
            contrib[u] ← 0;    dangling ← dangling + PR[u]
        else
            contrib[u] ← PR[u]/outdeg[u]
        end if
    end for
    base ← (1 − d)/N + d · dangling/N
    for all v ∈ V do
        PR′[v] ← base + d ∑_{u→v} contrib[u]
    end for
    ε ← ∑_v |PR′[v] − PR[v]|
    scambia PR e PR′
until ε < tolleranza
```

N, d	numero di nodi e damping factor: scalari, costanti per tutta l'esecuzione
PR$[v]$	punteggio corrente del nodo v; PR è un vettore di N componenti, una per nodo
PR$'[v]$	punteggio di v calcolato nell'iterazione in corso; vettore di appoggio, della stessa forma di PR
contrib$[u]$	contributo che u manda lungo ciascuno dei suoi archi uscenti; vettore di N componenti, ricalcolato ad ogni iterazione
outdeg$[u]$	numero di archi uscenti da u; vettore di N componenti, proprietà fissa del grafo
dangling	*dangling mass*: scalare, punteggio complessivo dei nodi senza archi uscenti
base	scalare, quota di punteggio che spetta a ogni nodo indipendentemente dagli archi
ε	scalare, variazione complessiva fra un'iterazione e la successiva

Ogni iterazione ricostruisce il vettore dei punteggi a partire da quello precedente, con due passaggi sui nodi. Nel primo ogni nodo è visto come sorgente: si calcola quanto manda lungo ciascun arco uscente e si raccoglie al tempo stesso la dangling mass. Nel secondo ogni nodo è visto come destinazione: si sommano i contributi che arrivano dai suoi predecessori. Si ripete finché la variazione complessiva non scende sotto la tolleranza. Il ciclo è il *metodo delle potenze*: si rimoltiplica il vettore finché non si stabilizza, e non serve normalizzarlo a ogni passo perché l'iterazione conserva da sé la somma a 1. *Pull* indica invece che ogni nodo legge dai predecessori anziché scrivere nei successori, scelta motivata nella Sezione 2.2.

Esplicitando *base*, il calcolo di PR$'[v]$ è esattamente l'iterazione (1):

$$\mathrm{PR}'[v] \;=\; \underbrace{\frac{1-d}{N} \;+\; d\,\frac{\mathrm{dangling}}{N}}_{base} \;+\; d \sum_{u\,:\,u \to v} \mathrm{contrib}[u]$$

dove contrib$[u]$ = PR$[u]$/outdeg$[u]$ per i nodi con archi uscenti, e contrib$[u] = 0$ per i dangling, il cui punteggio confluisce invece in dangling.

Il calcolo di contrib$[u]$ — il primo ciclo, quello dei *contributi* — è separato dal secondo, quello di *accumulo*, per una ragione di costo: eseguito così richiede N divisioni per iterazione anziché $M = |E|$, e nei grafi considerati $M \gg N$ (su `soc-LiveJournal1`, 69 milioni contro 4,8 milioni). La divisione è fra le operazioni aritmetiche più lente, quindi il risparmio è reale.

1.5 Necessità della parallelizzazione

La difficoltà del problema è interamente di scala. La matrice **MT** della Sezione 1.3 è *sparsa*: delle sue N^2 celle solo M sono diverse da zero, e nei grafi reali $M \ll N^2$. Rappresentarla come matrice densa $N \times N$ — una cella per ogni coppia (v, u) — richiederebbe una memoria quadratica in N e occupata quasi per intero da zeri, oltre a N^2 moltiplicazioni per iterazione anziché M. Si adotta perciò una rappresentazione *sparsa*, che conservi i soli elementi non nulli — il formato **CSR** della Sezione 2.2. Restano comunque M archi da percorrere a ogni iterazione, e le iterazioni si susseguono fino a convergenza: questo rende necessario distribuire il calcolo su più unità di elaborazione.

2 Metodologia adottata per la soluzione

2.1 Struttura del codice

Il progetto si compone di quindici moduli, dei quali tre soli formano il programma di calcolo vero e proprio: `src/pagerank.c` e `src/pagerank_cuda.cu`, che contengono il nucleo cronometrato

— il primo nelle versioni sequenziale e OpenMP, il secondo in quelle solo GPU e ibrida, e ogni eseguibile ne usa uno solo —, e `src/csr.c`, che porta il grafo in memoria prima che il cronometro parta. Gli altri preparano i dati, guidano l'esecuzione o servono a controllare i risultati.

Modulo	**Ruolo**
`tools/snap_to_csr.py`	conversione SNAP → CSR binario
`src/csr.c`, `csr.h`	caricamento e validazione
`src/pagerank.c`, `.h`	nucleo di calcolo sequenziale e OpenMP; interfaccia comune
`src/pagerank_cuda.cu`	nucleo di calcolo delle versioni solo GPU e ibrida
`src/main.c`	programma principale, output, log dei tempi
`src/csr_info.c`	statistiche di un grafo convertito
`tools/verify_pagerank.py`	verifica indipendente
`tools/run_benchmarks.sh`	misure su tutte le configurazioni
`tools/granularity.sh`	confronto fra il kernel ingenuo e le classi
`tools/plot_results.py`	figure di scalabilità
`tools/locality_stats.py`	statistiche di località delle letture sparse
`tools/nsys_phases.py`	fasi delle iterazioni ibride, dai profili `nsys`
`tools/hybrid_table.py`	tabella dei tempi della versione ibrida
`tools/plot_hybrid.py`	figura della versione ibrida
`tools/plot_degrees.py`	figura e tabella delle classi di righe

2.2 Rappresentazione dei dati: CSR

Per rappresentare i soli elementi non nulli della matrice **MT**, il formato **CSR** non usa una matrice bidimensionale $N \times N$ ma tre array monodimensionali:

- `col_idx[M]`: gli indici di colonna dei non nulli, letti riga per riga;
- `row_ptr[N+1]`: `row_ptr[v]` è la posizione, dentro `col_idx`, in cui comincia la riga v;
- `out_deg[N]`: il grado uscente di ogni nodo, cioè il denominatore outdeg(u) della (1).

La Figura 1 segue la costruzione su un grafo di cinque nodi e otto archi, dal grafo alla matrice agli array.

Tutto ruota attorno a `row_ptr`: gli elementi della riga v sono `col_idx[row_ptr[v] ... row_ptr[v+1]-1]`, e la lunghezza della riga è la differenza fra due offset consecutivi, che quindi non va memorizzata a parte. Nell'esempio della figura la riga 2 comincia in posizione `row_ptr[2]` $= 4$ e finisce prima di `row_ptr[3]` $= 7$: i predecessori del nodo 2 sono dunque `col_idx[4]`, `col_idx[5]` e `col_idx[6]`, cioè i nodi 0, 1 e 4 — come si può constatare nel grafo. Gli offset sono $N+1$ e non N proprio per questo motivo: la sentinella `row_ptr[N]` $= M$ fa sì che l'ultima riga si tratti come tutte le altre.

Perché manca l'array dei valori. In un CSR generico il terzo array è `val[M]`, con il valore di ogni elemento non nullo; qui al suo posto c'è `out_deg[N]`. Il motivo si legge nella Figura 1(b): tutti i non nulli della colonna u valgono 1/outdeg(u), così che il valore dipenda dalla sola colonna e sia ricavabile da `out_deg`. Ricavarlo a ogni accesso costerebbe però una divisione per arco: dentro `col_idx` ogni nodo u compare tante volte quanti sono i suoi archi uscenti, e ognuna rifarebbe la stessa divisione. Nell'esempio della Figura 1 il nodo 1 compare tre volte, una per ciascuno dei suoi tre archi uscenti, e il quoziente PR(1)/3 verrebbe ricalcolato identico tutte e tre le volte, perché né PR(1) né outdeg(1) cambiano durante l'iterazione. Il vettore `contrib` dell'Algoritmo 1 esegue invece la divisione una volta per nodo, in un passaggio separato che

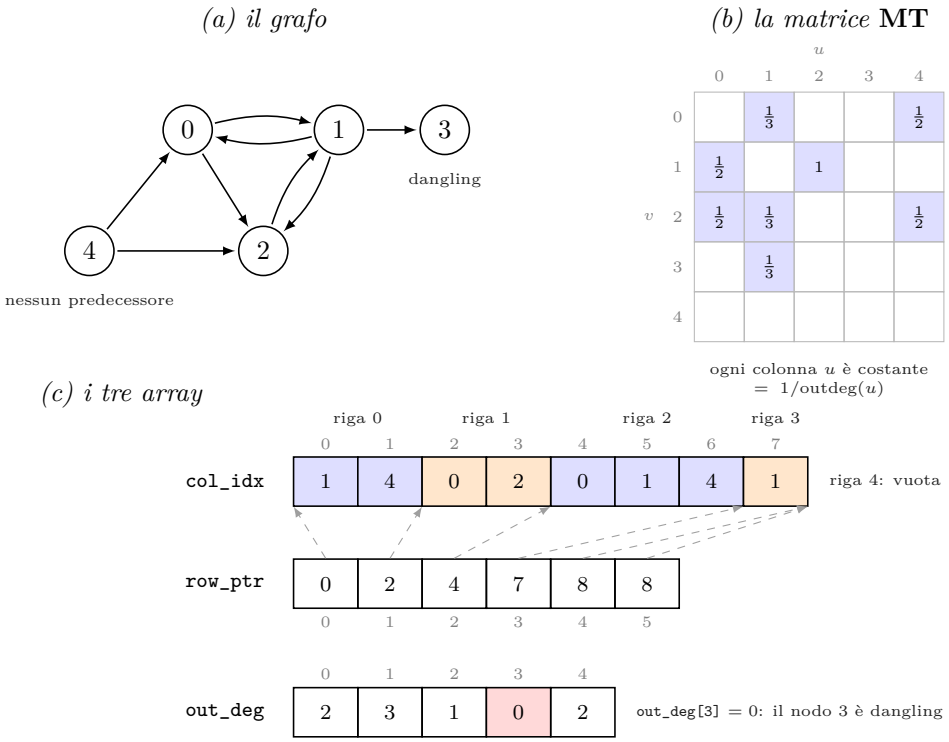


Figura 1: Costruzione del CSR del grafo trasposto, su un esempio con $N = 5$ nodi e $M = 8$ archi. In (b) la riga v della matrice **MT** raccoglie i *predecessori* di v, e ogni colonna u ha il valore costante $1/\text{outdeg}(u)$. In (c) `col_idx` concatena le righe di (b), `row_ptr[v]` indica dove ciascuna comincia e `out_deg` conserva i denominatori. Le due condizioni limite sono distinte e vivono su array distinti: la riga 4 è vuota perché nessuno punta al nodo 4, mentre `out_deg[3]` è nullo perché il nodo 3 non ha archi uscenti.

precede quello sui predecessori: a quest'ultimo resta solo da leggere `contrib[col_idx[j]]` e sommarlo, senza divisioni e senza valori da recuperare. Anche in memoria il guadagno viene dallo stesso passaggio da M a N, con $N \ll M$: su `soc-LiveJournal1` un `val[M]` in doppia precisione occuperebbe $8 \times M$ byte, cioè circa 526 MiB, mentre a `out_deg` bastano $4 \times N$ byte, circa 18,5 MiB: quasi trenta volte meno.

Perché il trasposto. Importante la scelta di quale indice diventa la riga. L'equazione (1) richiede, per calcolare $\mathrm{PR}_{t+1}(v)$, l'insieme dei nodi che puntano a v. Esistono due modi di organizzare il calcolo:

Push si scorre l'insieme degli archi $u \to v$ e si somma il contributo di u dentro $\mathrm{PR}'(v)$. Più archi convergono sullo stesso v, quindi più thread scrivono sulla stessa cella: si genera una *race condition* che richiede operazioni atomiche o lock.

Pull si scorre l'insieme dei nodi v, si leggono i suoi predecessori e si scrive $\mathrm{PR}'(v)$ una sola volta. Ogni nodo è di competenza di un solo thread: **non esiste alcuna scrittura concorrente**.

È stata adottata la formulazione *pull*, che richiede di memorizzare, per ogni nodo, la lista dei predecessori — cioè il grafo con tutti gli archi invertiti, il *trasposto*. È quanto illustra la Figura 1: la riga v raccoglie gli archi *entranti* in v, non quelli uscenti. L'inversione non richiede un passo dedicato: costruendo il CSR si sceglie semplicemente il nodo di destinazione, anziché quello di origine, come indice di riga.

Costo in memoria e modo di accesso. La struttura occupa $8\times(N+1)+4\times M+4\times N$ byte, ed è questa formula a produrre l'ultima colonna della Tabella 2: per `web-Google` 29,5 MiB contro i 5,6 TiB della matrice densa. Resta però un aspetto importante: il ciclo tocca tre array, e li tocca in due modi diversi. `row_ptr` e `col_idx` si leggono in ordine, $v = 0, 1, 2, \ldots$ e dentro ogni riga $j = 0, 1, 2, \ldots$; il processore se ne accorge e fa *prefetch*,[1] cosicché quando si arriva a quelle celle siano già in cache e l'attesa sia quasi nulla. Con `contrib[col_idx[j]]` non succede: l'indice arriva da `col_idx`, quindi all'interno di una riga i nodi sono ordinati in ordine crescente, ma la distanza tra un indice e il successivo è imprevedibile — nell'esempio della Figura 1, per la riga/nodo 0, prima `contrib[1]`, poi `contrib[4]` — e il prefetch non ha modo di indovinare quanto. Ogni accesso rischia così di aspettare la RAM, molto più lenta della cache.

Per ogni arco si va a prendere un contributo in una posizione imprevedibile e si esegue una sola somma: tanto traffico di memoria, pochissima aritmetica. In teoria il calcolo dovrebbe risultare *memory-bound*, e la Sezione 3.5 verifica se è davvero così.

2.3 Formato binario e pre-elaborazione

I grafi della collezione SNAP sono distribuiti come file di testo. Dopo l'intestazione seguono gli archi, uno per riga, come due interi separati da un carattere di tabulazione:

```
# Directed graph (each unordered pair of nodes is saved once): Wiki-Vote.txt
# Wikipedia voting on promotion to administratorship (till January 2008).
   ...
# Nodes: 7115 Edges: 103689
# FromNodeId    ToNodeId
30      1412
```

[1] Riconosciuto un andamento regolare negli accessi alla memoria, il processore porta in cache le celle successive prima che il programma le chieda. Funziona sugli accessi sequenziali; su quelli imprevedibili non ha nulla su cui basarsi.

```
30      3352
30      5254
```

Il primo intero è il nodo di partenza, il secondo quello di arrivo: ogni riga è dunque un arco orientato. Non c'è un elenco esplicito dei nodi — esistono quelli che compaiono in almeno un arco — e il formato non garantisce né che gli identificatori siano contigui, né che le righe siano ordinate, né che uno stesso arco non compaia due volte.

La conversione dal formato testuale SNAP al CSR avviene *una volta sola*, offline: l'analisi del testo è lenta e irregolare, ed eseguirla dentro il programma misurato significherebbe misurare il parser anziché l'algoritmo. Si tratta di sola preparazione dei dati e il risultato è l'input del programma vero e proprio, non una sua parte: il convertitore `tools/snap_to_csr.py` è scritto in Python per comodità, dato che numpy risolve in poche righe la rimappatura degli identificatori e l'ordinamento degli archi.

Gli identificatori SNAP sono sparsi (`web-Google` ha 875 713 nodi con id fino a 916 428) e vengono rimappati su un intervallo contiguo $[0, N)$; la corrispondenza inversa è salvata in un file `.ids` separato, letto solo per riportare i risultati negli identificatori originali.

Per conservare il risultato si è scelto un formato binario invece che testuale. Due ragioni a favore:

- *velocità*: leggere il binario è solo copiare byte,[2] perché i byte del file sono già nella forma in cui il programma li usa, senza niente da interpretare, mentre in testo servirebbe un parser in C che a ogni esecuzione rilegga e riconverta i numeri cifra per cifra;
- *dimensione*: in binario un numero occupa i byte del suo tipo, in testo un carattere per cifra più il separatore.

E due prezzi da pagare:

- *leggibilità*: il file non si apre con un editor per dargli un'occhiata; per guardare dentro un grafo convertito serve uno strumento apposta, ed è `src/csr_info.c`;
- *errori silenziosi*: qualunque sequenza di byte può essere un CSR valido, sicché un file corrotto o troncato viene letto lo stesso e produce risultati sbagliati senza avvisare. Un testo malformato invece è più facile da rilevare: chi lo legge trova un carattere dove si aspettava una cifra, e si ferma. È per questo che `src/csr.c` ha il magic number e i cinque controlli della Sezione 2.5.

Il grafo è scritto in un file `.csr`, nel formato binario seguente:

Campo	**Tipo**	**Dimensione**
magic `"PRCSR001"`	`char[8]`	8 byte
`n_nodes`	`uint64`	8 byte
`n_edges`	`uint64`	8 byte
`row_ptr`	`uint64[]`	$8(N{+}1)$ byte
`col_idx`	`uint32[]`	$4M$ byte
`out_deg`	`uint32[]`	$4N$ byte

I primi otto byte identificano il formato: il caricatore li confronta e rifiuta subito un file che non sia un `.csr`, invece di interpretarne i primi byte come dimensioni. Le tre cifre finali sono un numero di versione.

Per ciascuno dei tre array resta da decidere quanti byte dare a ogni singolo numero, 4 oppure 8. Servono due informazioni, ed è bene tenerle distinte: *quante celle* ha l'array, che dice quanto si

[2] Il caricamento si riduce a una `fread` per array.

guadagna a stringerlo, e *quanto valgono* i numeri che vi finiscono dentro, che dice se stringerlo è possibile. In 4 byte sta qualunque valore fino a circa 4,3 miliardi.

Array	**Celle**	**Contenuto di una cella**	**Valore massimo**
`col_idx`	M, una per arco	l'indice di un nodo	$N-1$
`row_ptr`	$N+1$, una per nodo	una posizione dentro `col_idx`	M
`out_deg`	N, una per nodo	il grado uscente del nodo	$N-1$

Le prime due righe sono incrociate: `col_idx` ha molte celle e valori piccoli, `row_ptr` poche celle e valori grandi. Su `soc-LiveJournal1` le celle sono 69 milioni (archi) contro 4,8 milioni (nodi).

`col_idx` usa perciò 32 bit. Si può, perché dentro ci sono indici di nodo e il più alto è solo 4 847 571. E conviene, perché le celle sono M: i 4 byte risparmiati su ognuna valgono 263 MiB su un file che in tutto ne pesa 318,7. `row_ptr` resta invece a 64 bit. Stringerlo sarebbe possibile — 69 milioni in 4 byte ci sta — ma rende poco e costa un limite. Rende poco perché le celle sono quattordici volte meno: 18 MiB invece di 263. Costa un limite perché dentro ci sono posizioni nell'array degli archi, che crescono velocemente con M. `out_deg` sta a 32 bit come `col_idx`, perché ha un valore massimo che non supera N, e in più la scelta pesa poco perché, come `row_ptr`, ha N celle. Il tipo stretto va insomma dove le celle sono tante e i valori piccoli, quello largo dove le celle sono poche e i valori possono crescere.

Il guadagno atteso, poi, non è tanto sul disco quanto sul traffico di memoria: per ogni arco il ciclo legge un elemento di `col_idx` ed esegue una sola moltiplicazione-somma, cosicché portare ogni cella da 8 a 4 byte dimezza i byte letti in sequenza.

2.4 Precisione: `float` contro `double`

Bisogna decidere anche quanti byte dare a ogni numero dei tre vettori dell'Algoritmo 1 — PR, PR$'$ e contrib — che non fanno parte del `.csr` ma vivono solo in memoria, e condividono un unico tipo, definito da un `typedef` che si può cambiare in fase di compilazione con `-DPAGERANK_FLOAT`. La motivazione è che sulle GPU di fascia consumer l'aritmetica FP32 (`float`, 4 byte) è nettamente più veloce della FP64 (`double`, 8 byte). Mantenere entrambe le varianti raggiungibili dallo stesso sorgente permette di confrontarne accuratezza e velocità.

Gli accumulatori — `accum_t` nel codice — restano invece sempre in `double`. Sono tre sole variabili scalari: la somma dei contributi di un nodo, la massa dangling e la variazione ε fra due iterazioni. Tenerle larghe non costa quindi traffico di memoria, mentre stringerle costerebbe accuratezza.

2.5 Validazione del file in ingresso

Il caricatore in `src/csr.c` verifica cinque proprietà del file prima di dichiarare riuscito il caricamento:

- `check_header_counts` — i conteggi dichiarati sono compatibili con la dimensione del file;
- `check_header_size` — il file misura esattamente quanto l'intestazione implica;
- `check_rowptr_span` — i due estremi di `row_ptr`: il primo offset vale 0, l'ultimo vale M;
- `check_rowptr_monotonic` — gli offset non decrescono, il che insieme al controllo precedente li confina tutti in $[0, M]$;

- `check_neighbours_are_existing_nodes` — ogni vicino memorizzato è un nodo esistente.

I controlli fatti in fase di caricamento permettono al programma di assumere che i dati siano corretti e di non verificare nulla dentro i cicli di calcolo (`pagerank.c`).

2.6 Implementazione sequenziale e OpenMP

Le due versioni condividono lo stesso file sorgente `pagerank.c` poiché le direttive OpenMP sono ignorate dal compilatore quando non gli si passa `-fopenmp`, che produce quindi un programma sequenziale. Oltre a non esistere così duplicazione di codice che possa divergere, il riferimento sequenziale è *onesto*: misurando la versione parallela limitata a un thread si otterrebbe un tempo peggiore (Tabella 4 e Tabella 5, colonna a un thread, sette grafi su otto su entrambe le macchine), perché si pagherebbe comunque la creazione del team di thread: usarla come riferimento gonfierebbe silenziosamente ogni speed-up dichiarato.

Il nucleo contiene due cicli paralleli per iterazione.

```
#pragma omp parallel for schedule(static) reduction(+ : dangling)
for (v = 0; v < n; v++) {
    if (g->out_deg[v] == 0) {
        contrib[v] = 0;
        dangling  += cur[v];
    } else {
        contrib[v] = cur[v] / g->out_deg[v];
    }
}
```

Listing 1: Calcolo dei contributi e massa dangling.

La clausola `reduction(+ : dangling)` assegna a ogni thread una copia privata della variabile, inizializzata a zero, e ne somma i valori al termine del ciclo. È la soluzione migliore al problema dell'aggiornamento concorrente: una variabile condivisa protetta da lock serializzerebbe l'intero ciclo. Lo scheduling è `static` perché il lavoro per iterazione è costante.

```
#pragma omp parallel for schedule(dynamic, 256) reduction(+ : error)
for (v = 0; v < n; v++) {
    accum_t sum = 0.0;
    for (j = g->row_ptr[v]; j < g->row_ptr[v + 1]; j++)
        sum += contrib[g->col_idx[j]];
    nxt[v] = base + d * sum;
    error += fabs(nxt[v] - cur[v]);
}
```

Listing 2: Accumulo dai predecessori (CSR).

La clausola `reduction` su `error` funziona allo stesso modo, mentre lo scheduling è `dynamic`: la lunghezza delle righe varia di ordini di grandezza, quindi una suddivisione statica assegnerebbe a ogni thread lo stesso *numero di righe* ma non la stessa *quantità di lavoro*, lasciando inattivi i thread che hanno ricevuto solo nodi di grado basso. La dimensione del chunk (256) è un parametro che va calibrato provando.

Alternanza dei buffer. L'iterazione alterna due vettori (`rank` e `next`) scambiando due puntatori (`cur`, che indica sempre i punteggi più recenti, e `nxt`, dove si scrivono quelli nuovi) anziché copiare i dati; una sola `memcpy` finale è necessaria, e solo se il numero di scambi è dispari.

2.7 Implementazione ibrida OpenMP + CUDA

La versione ibrida sta in un solo file, `src/pagerank_cuda.cu`, che nella build ibrida prende il posto di `pagerank.c`: definisce la stessa funzione `pagerank()`, così che `main.c` e `csr.c` restino quelli delle altre versioni. Lo compila `nvcc`, che passa `-fopenmp` al compilatore dell'host per la parte di lavoro che resta alla CPU. I due cicli dell'Algoritmo 1 diventano kernel CUDA, e una parte delle righe del secondo, le più lunghe, le calcolano i thread OpenMP. Con l'opzione `-s 0` la CPU non riceve righe: è la versione solo GPU, lo stesso codice senza copie in più a ogni iterazione e senza avviare i thread OpenMP. Per le chiamate CUDA viene usata la macro `HANDLE_ERROR`, che ferma il programma al primo errore indicandone il punto.

La Figura 2 riassume le fasi del programma e, in dettaglio, la preparazione che precede il cronometro.

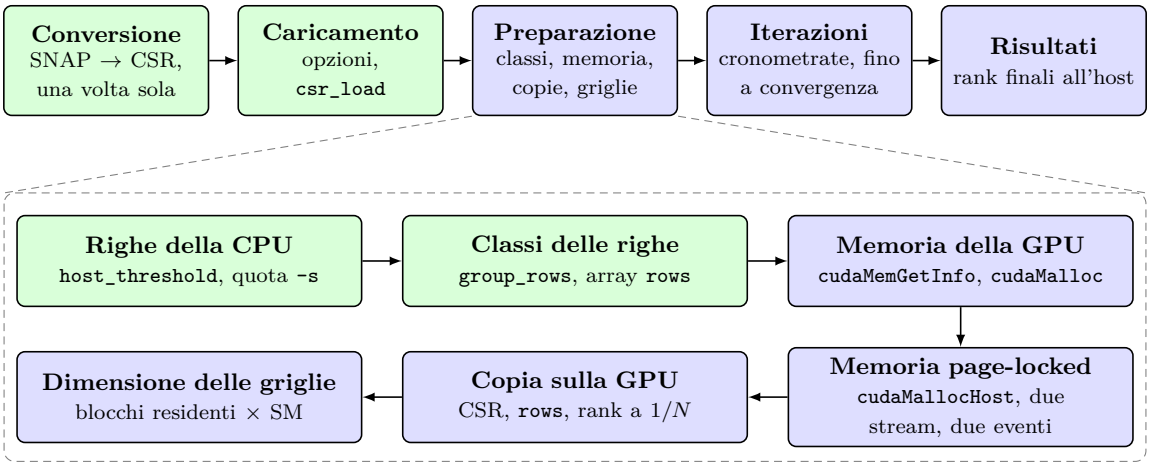


Figura 2: Le fasi della versione ibrida, dalla conversione del grafo ai risultati, e sotto, in dettaglio, la preparazione che precede il cronometro: solo il ciclo delle iterazioni è cronometrato. Nella preparazione, sull'host si sceglie quali righe vanno alla CPU e si raggruppano le altre per classe; poi si controlla e si alloca la memoria, si copiano sulla GPU i tre array del CSR, l'array `rows` delle righe raggruppate per classe e i rank iniziali, e si dimensionano le griglie dei kernel (ciascuna con al più tanti blocchi quanti ne possono risiedere contemporaneamente in un SM, moltiplicati per il numero di SM). In verde il lavoro del solo host, in blu quello che coinvolge la GPU.

Il grafo sulla GPU. I tre array del CSR e i vettori `cur`, `nxt` e `contrib` sono allocati in memoria globale con `cudaMalloc`. Il grafo viene copiato una volta sola, prima che il cronometro parta, e i rank tornano all'host una volta sola, dopo che si è fermato (Sezione 3.4); fra un'iterazione e l'altra i due vettori dei rank si alternano scambiando i puntatori passati ai kernel, come nella versione per CPU. Prima di allocare, il programma chiede al runtime la memoria libera (`cudaMemGetInfo`) e la confronta con quella che servirà (se non basta si ferma dicendo quanta ne serve).

Il primo ciclo e le riduzioni. Il calcolo dei contributi diventa `contrib_kernel`. Come tutti i kernel di `pagerank_cuda.cu`, usa un ciclo *grid-stride*. La dangling mass si calcola con una riduzione, come nella versione OpenMP: ogni thread somma la sua parte in un registro, i 32 thread di un warp combinano le loro con `__shfl_down_sync`, gli 8 warp di un blocco attraverso la shared memory, dopo una barriera `__syncthreads`, e infine ogni blocco aggiunge il suo totale a una variabile in memoria globale con un solo `atomicAdd`: uno per blocco e non per thread, perché migliaia di aggiornamenti sullo stesso indirizzo verrebbero serializzati.

Il secondo ciclo: tre kernel per tre classi. Il rank nuovo di un nodo richiede la somma dei contributi dei suoi predecessori, cioè degli elementi della sua riga, e la scelta è quanti thread dedicare a ciascuna riga. Un thread per riga, la traduzione diretta del ciclo OpenMP, va bene finché le righe sono corte e di lunghezza simile: i 32 thread di un warp procedono al passo del più lento, e una riga lunga trattiene tutto il suo warp; affidata a un solo thread, una riga di centinaia di migliaia di predecessori allungherebbe l'intero kernel. Le righe sono perciò divise in tre classi, ciascuna con il suo kernel:

- fino a T predecessori, `accumulate_thread_rows`: un thread per riga;
- fino a W, `accumulate_warp_rows`: un warp per riga, in cui il thread k legge gli elementi k, $k+32$, $k+64$, ... della riga. I 32 thread leggono così insieme 32 elementi consecutivi di `col_idx`, con accesso coalescente. Le 32 somme parziali si sommano poi dentro il warp con `__shfl_down_sync`;
- oltre W, `accumulate_block_rows`: un blocco di 256 thread per riga, con lo stesso schema; le 256 somme parziali si sommano prima dentro ogni warp con `__shfl_down_sync`, poi fra gli 8 warp attraverso la shared memory.

Le soglie si scelgono a tempo di esecuzione con l'opzione `-b T,W`, e valgono 16 e 256 di default (Tabella 1). Una T oltre la riga più lunga dà a ogni riga un thread. Prima del ciclo le righe vengono raggruppate per classe in un array `rows`, e ciascun kernel ne riceve la sua porzione. I nodi conservano però la loro numerazione: rinumerarli per classe disperderebbe i predecessori che l'ordine originale tiene vicini, cioè la località che la Sezione 3.5 ha mostrato decisiva. I kernel sono accodati sullo stesso stream, quindi vengono eseguiti uno dopo l'altro e leggono `contrib` quando `contrib_kernel` è terminato.

Perché una granularità variabile. La distribuzione delle lunghezze di riga, misurata da `csr_info.c`, è riportata in Tabella 1, e le sue due metà dicono cose opposte. La tabella la divide nelle tre classi in cui la GPU mette le righe con le soglie di default: fino a 16 predecessori, da 17 a 256 e oltre 256. Nei grafi utilizzati fra il 67 e il 96 % delle righe ha al più 16 elementi, ma quelle righe valgono, `cit-Patents` a parte, dal 5 al 44 % degli archi; le righe oltre i 256 elementi sono al più lo 0,6 % e ne raccolgono fino al 46 %, con singole righe che arrivano a centinaia di migliaia di predecessori. Le righe da 17 a 256 elementi portano invece il grosso del lavoro sui grafi sociali, l'81 % degli archi su `soc-Pokec`. Fa eccezione `cit-Patents`, che non ha hub: tre quarti dei suoi archi stanno in righe fino a 16 elementi, e quelle oltre 256 ne raccolgono appena lo 0,1 %. Questi numeri giustificano la scelta descritta sopra. Con un thread per riga, una sola riga di centinaia di migliaia di predecessori fermerebbe il suo warp per altrettanti passi. Con le classi, nessun thread aspetta gli altri del suo warp per più di 16 passi, e ogni warp o blocco lavora con almeno metà dei suoi thread. La Sezione 3.7 lo verifica sperimentalmente (Tabella 9).

La Figura 3 mostra la stessa distribuzione nodo per nodo. Il numero di nodi cala di ordini di grandezza al crescere dei predecessori, ma la coda arriva a migliaia o a decine di migliaia di predecessori, oltre i 100 000 su `wiki-topcats`; solo `cit-Patents` si ferma sotto i mille. Nella coda molti valori hanno un solo nodo, e i loro punti si allineano sull'1: sono gli hub. Su `soc-Pokec` la curva si piega presto verso il basso, e pochissimi nodi superano il migliaio di predecessori.

La quota della CPU. Si prevede di dare alla CPU i nodi di grado più alto, gli hub, pochi e lunghi, adatti alla bassa latenza dei suoi core, e alla GPU le righe corte, numerose e simili fra loro, adatte al suo throughput. La ripartizione fra CPU e GPU si chiede con l'opzione `-s`, la *quota*: la frazione degli archi, fra 0 e 1, da assegnare alla CPU; con `-s 0.25` si chiede

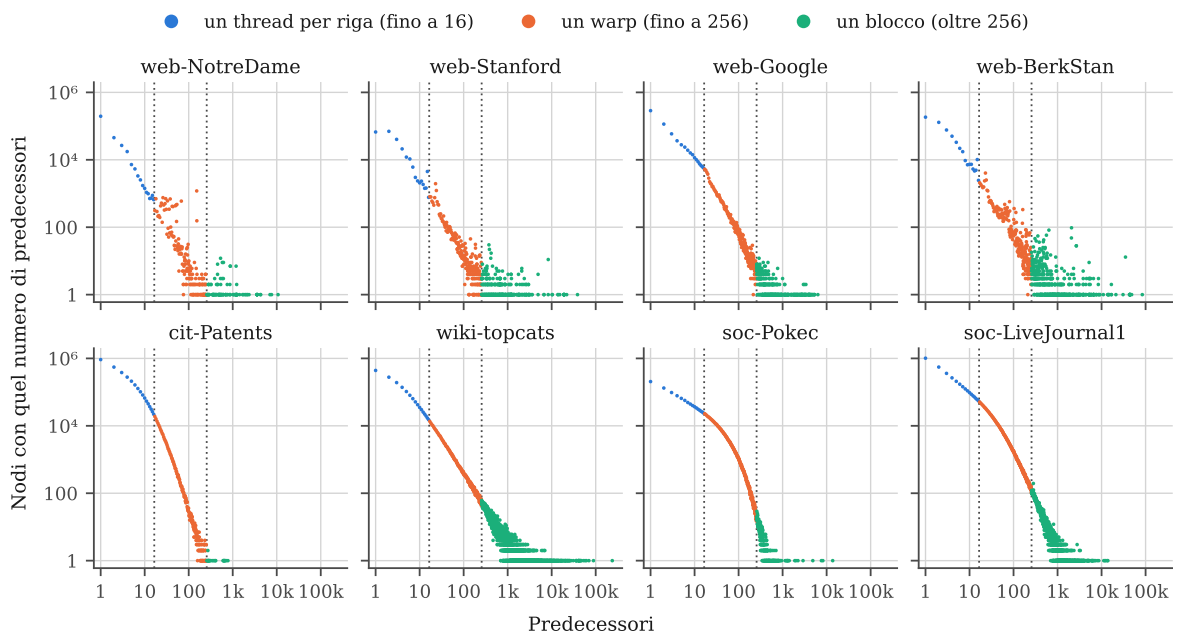


Figura 3: Nodi per numero di predecessori, cioè per lunghezza della loro riga nel CSR trasposto, su assi logaritmici: un punto per ogni numero di predecessori presente nel grafo. Il colore è la classe in cui la GPU mette quelle righe con le soglie di default, 16 e 256, segnate dalle linee punteggiate (Tabella 1). I nodi senza predecessori non compaiono. Figura generata da `tools/plot_degrees.py`.

per la CPU un quarto degli archi. La quota 1 non equivale alla versione OpenMP: anche con tutte le righe alla CPU, `contrib_kernel` resta sulla GPU, e a ogni iterazione `contrib` passa all'host e i rank tornano alla GPU. `host_threshold` parte dalle righe più lunghe e scende una lunghezza alla volta, prendendo ogni volta tutte le righe di quella lunghezza, finché gli archi raccolti non supererebbero la quota chiesta; l'ultima lunghezza presa è la lunghezza minima, e alla CPU andranno tutte le righe con almeno quel numero di predecessori. Una soglia fissa sul numero di predecessori non sarebbe portabile fra i grafi: con `-s 0.25` la lunghezza minima è 18 su `cit-Patents` e 2 676 su `web-BerkStan`. Prendendo lunghezze intere, la quota ottenuta può restare sotto quella chiesta, e di molto dove una sola lunghezza raccoglie molti archi: su `web-NotreDame`, `-s 0.25` dà alla CPU il 14,6 % degli archi. Le righe della CPU sono l'ultima classe dell'array `rows`.

Un'iterazione, fra due stream. Per calcolare le sue righe la CPU ha bisogno di `contrib` per intero, perché i predecessori di una riga possono essere ovunque nel grafo. A ogni iterazione tutto il vettore deve quindi passare dalla GPU all'host, qualunque sia la quota; la CPU, a sua volta, deve rimandare i rank delle sue righe. Perché questi trasferimenti non fermino la GPU, l'iterazione usa due stream e due eventi, nell'ordine della Figura 4:

1. sullo stream principale `contrib_kernel` calcola i contributi e la dangling mass;
2. un secondo stream, che con `cudaStreamWaitEvent` attende il solo `contrib_kernel`, copia `contrib` e la dangling mass sull'host con `cudaMemcpyAsync`, mentre lo stream principale prosegue con i kernel di accumulo delle righe della GPU: la copia la esegue il *copy engine*, e si sovrappone al calcolo;
3. l'host attende la fine della copia (`cudaEventSynchronize`) e i thread OpenMP calcolano le righe della CPU, con `schedule(dynamic, 16)`, perché anche queste hanno lunghezze

Tabella 1: Distribuzione delle lunghezze di riga (predecessori per nodo) nelle tre classi della GPU, con le soglie di default: quota percentuale di nodi e di archi per classe. Le righe vuote stanno nella classe dei thread. Valori misurati da `csr_info.c`.

	thread (0–16)		**warp (17–256)**		**blocco (>256)**		
Grafo	nodi	archi	nodi	archi	nodi	archi	**max**
wiki-Vote	76,4	5,2	23,5	91,5	0,2	3,3	457
web-NotreDame	95,8	43,1	4,2	45,2	0,1	11,7	10 721
web-Stanford	94,7	35,0	5,1	24,3	0,2	40,6	38 606
web-Google	93,1	43,5	6,8	43,8	0,1	12,7	6 326
web-BerkStan	93,3	25,9	6,4	28,4	0,3	45,7	84 208
cit-Patents	95,8	74,0	4,2	25,9	0,0	0,1	779
wiki-topcats	86,7	22,5	12,7	37,9	0,6	39,6	238 040
soc-Pokec	66,7	17,7	33,3	80,8	0,1	1,5	13 733
soc-LiveJournal1	78,8	22,8	20,9	66,8	0,3	10,4	13 906

molto diverse, e `reduction(+ : error)`;

4. i nuovi rank della CPU, scritti uno dopo l'altro nell'ordine di `rows`, tornano alla GPU con una `cudaMemcpyAsync`, accodata sullo stream principale dopo i kernel di accumulo, e il kernel `scatter_host_ranks` li riporta al loro posto in `nxt`, prima del `contrib_kernel` dell'iterazione successiva;
5. l'ultima copia dello stream riporta sull'host l'errore della GPU, e l'host attende la fine dell'iterazione con `cudaStreamSynchronize`.

Le copie asincrone richiedono memoria dell'host *page-locked*, l'unica che la GPU può leggere e scrivere da sé mentre esegue i kernel: il buffer che riceve `contrib` e quello dei rank della CPU sono allocati con `cudaMallocHost`. Gli eventi servono solo a sincronizzare: sono creati con `cudaEventDisableTiming`, che evita di registrare l'istante in cui avvengono, qui inutile.

L'errore di convergenza. I kernel di accumulo sommano anche la variazione $|\mathrm{PR}'[v]-\mathrm{PR}[v]|$ delle loro righe, con lo stesso meccanismo usato nel primo ciclo per la riduzione della dangling mass; l'host somma a questo totale la variazione delle righe della CPU. Dangling mass ed errore della GPU stanno in un array di due scalari in memoria globale, azzerato a inizio iterazione e riportato sull'host alla fine con una sola copia: con la GPU sola, questi 16 byte sono l'unico trasferimento di ogni iterazione.

Nessuna ipotesi sul dispositivo. I blocchi hanno 256 thread, un multiplo della dimensione del warp. Il numero di blocchi di ogni kernel è quello che la GPU tiene residenti tutti insieme: i blocchi per multiprocessore, che il runtime calcola con `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, per il numero di multiprocessori, letto con `cudaGetDeviceProperties`; meno, se il lavoro ne richiede meno. Con i cicli grid-stride questi stessi blocchi trattano più nodi al crescere del grafo, senza che il loro numero aumenti. La prima chiamata CUDA del programma, che avvia il driver sulla GPU e richiede un tempo non trascurabile, è quella che descrive il dispositivo, e `main.c` la esegue prima del cronometro. Il `Makefile`, infine, compila per l'architettura della GPU presente (`-arch=native`), ma è sovrascrivibile.

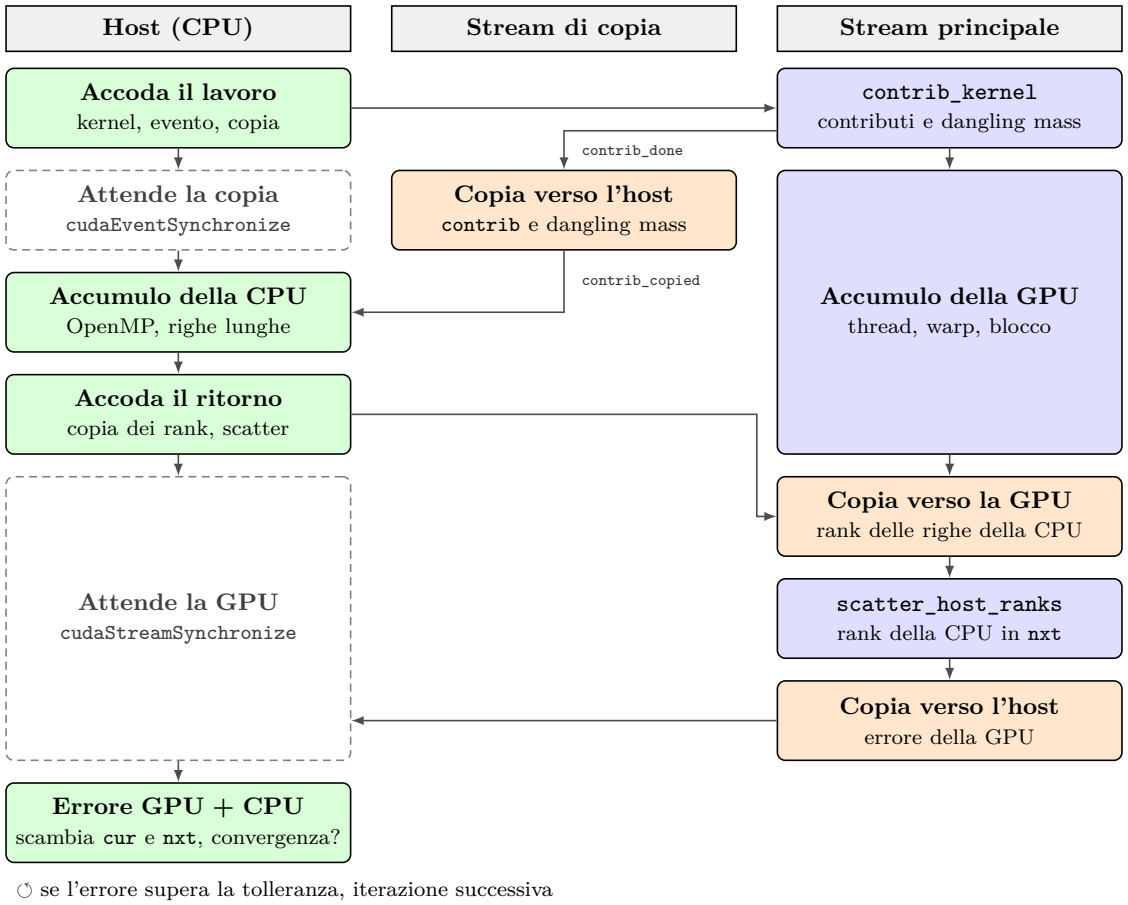


Figura 4: Un'iterazione della versione ibrida in dettaglio, con il tempo verso il basso: una colonna per l'host e una per ciascuno stream. In blu i kernel della GPU, in arancio le copie, in verde il lavoro dell'host, tratteggiate le attese. Con `-s 0` sparisce lo stream di copia; nello stream principale mancano "Copia verso la GPU" e lo scatter dei rank, e l'host attende solo la GPU alla fine dell'iterazione, per leggerne l'errore e decidere se continuare.

2.8 Strategia di verifica

Il programma produce milioni di valori, non ispezionabili a occhio. La correttezza è quindi stabilita ricalcolando il risultato per vie che non condividono codice con l'implementazione sotto esame (`tools/verify_pagerank.py`):

1. PageRank viene ricalcolato su una **matrice densa** $N \times N$, applicando l'iterazione (3) con un solo prodotto matrice-vettore anziché con i due cicli del nucleo di calcolo; il codice è scritto da me;
2. il riferimento denso viene confrontato con **NetworkX**; il codice è di terze parti;
3. si verifica infine che il risultato del **programma C** coincida con il riferimento. Prima del confronto si verificano due condizioni sul risultato stesso: che il programma sia giunto a convergenza e che la somma dei rank sia 1.

Un errore nella conversione da testo a CSR (`tools/snap_to_csr.py`) non sfugge: i risultati dei riferimenti (matrice densa e NetworkX) nascono dall'edge list originale (`.txt`), mentre il programma C calcola dal `.csr`: se i due non corrispondessero, il confronto metterebbe a paragone il PageRank di due grafi diversi e i valori non coinciderebbero.

Poiché la matrice densa richiede N^2 elementi, il punto 1 è possibile solo sul grafo più piccolo: 386 MiB per `wiki-Vote` contro i 5,6 TiB che servirebbero per `web-Google`. È questa la ragione per cui `wiki-Vote` fa parte dell'insieme di test. Sui quattro grafi web il confronto del programma C con NetworkX è l'unico possibile.

Il controllo è quindi svolto su `wiki-Vote` e sui quattro grafi web, e queste istanze vengono ritenute sufficienti per stabilire che il codice è corretto.

Sulle quattro istanze maggiori il confronto con NetworkX non è praticabile. NetworkX rappresenta il grafo con dizionari Python, a circa 500 byte per arco: per `soc-LiveJournal1`, con 69 milioni di archi, servirebbero più di 30 GiB.

Il controllo viene svolto sia su tutti i valori sia sull'ordine dei primi k nodi (default 100). Quello sull'ordine ha senso solo in testa: più in giù i punteggi si addensano fino a coincidere e l'ordine fra pari dipende solo da come l'ordinamento spezza i pareggi. Uno scambio viene perciò segnalato come errore soltanto se avviene fra valori effettivamente distinti.

Il controllo di convergenza al punto 3 è stato implementato perché il riferimento e il programma ricevono lo stesso tetto di iterazioni, e se lo si abbassa entrambi si fermano allo stesso punto, con valori che coincidono pur essendo lontani dall'essersi stabilizzati. Il confronto avrebbe allora successo su un risultato quantitativamente sbagliato, accettandolo come corretto. Per la stessa ragione anche il riferimento denso viene scartato se non giunge a convergenza.

3 Risultati sperimentali

3.1 Ambienti di prova

	Macchina 1
Sistema operativo	Ubuntu 22.04.5 LTS, kernel Linux 6.8
CPU	Intel Core i5-9300H, 4 core fisici / 8 thread, 2,40–4,10 GHz
Cache	L1d 128 KiB, L2 1 MiB, L3 8 MiB
RAM	16 GiB
GPU	NVIDIA GeForce GTX 1050 Ti, 4 GiB, compute capability 6.1, 6 SM
Compilatore	`gcc`, OpenMP 4.5; CUDA Toolkit 12.8
Opzioni	`-std=c11 -Wall -Wextra -O2`
	Macchina 2
Sistema operativo	Ubuntu 20.04.6 LTS, kernel Linux 5.15
CPU	Intel Core i7-6700, 4 core fisici / 8 thread, 3,40–4,00 GHz
Cache	L1d 128 KiB, L2 1 MiB, L3 8 MiB
RAM	32 GiB
GPU	NVIDIA GeForce RTX 2080 Ti, 11 GiB, compute capability 7.5, 68 SM NVIDIA GeForce GTX 1080 Ti, 11 GiB, compute capability 6.1, 28 SM
Compilatore	`gcc` 9.4, OpenMP 4.5; CUDA Toolkit 11.8
Opzioni	`-std=c11 -Wall -Wextra -O2`
	Macchina 3
Sistema operativo	Ubuntu 24.04.4 LTS, kernel Linux 6.8
CPU	Intel Xeon W-3323, 12 core fisici / 24 thread, 3,50–3,90 GHz
Cache	L1d 576 KiB, L2 15 MiB, L3 21 MiB
RAM	64 GiB
GPU	NVIDIA GeForce RTX 2080 Ti, 11 GiB, compute capability 7.5, 68 SM NVIDIA GeForce RTX 2080 Ti, 11 GiB, compute capability 7.5, 68 SM
Compilatore	`gcc` 13.3, OpenMP 4.5; CUDA Toolkit 13.1
Opzioni	`-std=c11 -Wall -Wextra -O2`

La versione OpenMP è stata misurata sulle Macchine 1 e 3: la Macchina 3 ha il triplo dei core fisici e una L3 di 21 MiB anziché 8. Non è stata misurata sulla Macchina 2, la cui CPU ha la stessa struttura di quella della Macchina 1 (4 core fisici, 8 thread logici, cache delle stesse dimensioni): non avrebbe aggiunto un caso nuovo allo studio della scalabilità, che dipende dal numero di core fisici.

La versione ibrida è stata misurata su tutte e tre le macchine, una volta per ogni GPU: cinque configurazioni in tutto. Le GPU non sono collegate tutte allo stesso modo: lavorano su PCIe 3.0 x16 la GTX 1050 Ti della Macchina 1 e la prima RTX 2080 Ti della Macchina 3, su x8 la seconda RTX 2080 Ti della Macchina 3 ed entrambe le GPU della Macchina 2; la differenza conta quando la CPU ha una parte del lavoro (Sezione 3.7).

3.2 Istanze di prova

Sono stati usati nove grafi reali della collezione SNAP [2]: una rete di voti fra utenti di Wikipedia, quattro reti web (tre di pagine di domini universitari e una diffusa da Google), una rete di citazioni fra brevetti, una di collegamenti fra voci di Wikipedia e due reti sociali.

Ogni istanza ha un ruolo. `wiki-Vote` è troppo piccola perché i tempi siano significativi e resta nell'insieme come riferimento di correttezza (Sezione 2.8).

Tabella 2: Istanze utilizzate.

Grafo	Nodi	Archi	Dangling	max in-deg	CSR
wiki-Vote	7 115	103 689	1 005	457	0,5 MiB
web-NotreDame	325 729	1 497 134	187 788	10 721	9,4 MiB
web-Stanford	281 903	2 312 497	172	38 606	12,0 MiB
web-Google	875 713	5 105 039	136 259	6 326	29,5 MiB
web-BerkStan	685 230	7 600 595	4 744	84 208	36,8 MiB
cit-Patents	3 774 768	16 518 948	1 685 423	779	106,2 MiB
wiki-topcats	1 791 489	28 511 807	0	238 040	129,3 MiB
soc-Pokec	1 632 803	30 622 564	200 110	13 733	135,5 MiB
soc-LiveJournal1	4 847 571	68 993 773	539 119	13 906	318,7 MiB

I quattro grafi web, di taglia simile (da 1,5 a 7,6 milioni di archi), formano due coppie che differiscono per la numerazione dei nodi: in `web-NotreDame` e `web-BerkStan` i predecessori di un nodo hanno indici vicini, in `web-Stanford` e `web-Google` sono abbastanza lontani da richiedere quasi a ogni arco una linea di cache nuova (Tabella 6). Così l'effetto della località si osserva su due casi per parte, e non su uno solo.

Le quattro istanze maggiori, da 16,5 a 69 milioni di archi, hanno tutte un vettore `contrib` in doppia precisione più grande della cache L3 della Macchina 1, e due anche di quella della Macchina 3 (Tabella 6). Ciascuna ha poi una sua particolarità: `cit-Patents` non ha hub (nessun brevetto è citato più di 779 volte) e ha le righe più corte, 4,4 predecessori in media; `wiki-topcats` ha l'hub più estremo, con 238 040 predecessori, e nessun nodo dangling; `soc-Pokec` e `soc-LiveJournal1` hanno la quota più alta di righe lunghe, oltre 16 predecessori per il 33,4 % e il 21,2 % dei nodi (Tabella 1).

3.3 Correttezza

La Tabella 3 raccoglie i confronti fra i tre calcoli della Sezione 2.8: il riferimento denso, NetworkX e il programma C. Lo scarto è la maggiore differenza in valore assoluto fra i rank corrispondenti, cercata su tutti gli N nodi. I confronti usano la build sequenziale; ripetuti con quella OpenMP danno lo stesso scarto. Lo stesso vale per la build ibrida; sulle quattro istanze maggiori, dove NetworkX non è praticabile, la versione ibrida coincide con quella OpenMP entro $5 \cdot 10^{-17}$, nello stesso numero di iterazioni, in esecuzioni di verifica apposite, distinte da quelle cronometrate, alle quote 0, 0,4 e 1.

Tabella 3: Confronto fra le tre implementazioni.

Confronto	Istanza	Scarto	Esclude
NetworkX contro denso	`wiki-Vote`	$4{,}5 \cdot 10^{-12}$	fraintendimenti dell'algoritmo
C contro denso	`wiki-Vote`	$7{,}1 \cdot 10^{-14}$	errori nella rappresentazione dei dati
C contro NetworkX	grafi web	$3{,}6 \cdot 10^{-10}$	errori su grafi grandi

Lo scarto contro NetworkX riportato in tabella è quello di `web-NotreDame`, il più alto dei quattro grafi web; sugli altri tre resta sotto $6{,}5 \cdot 10^{-11}$. In ogni caso è ben al di sotto della soglia di 10^{-8} usata dallo strumento di verifica.

In tutte le 2 295 esecuzioni registrate nei `bench.csv` delle tre macchine, ibride comprese, la somma dei rank si discosta da 1 per non più di $1{,}4 \cdot 10^{-11}$ in doppia precisione e $2{,}7 \cdot 10^{-8}$ in singola: la massa dei nodi dangling non si disperde. Tutte giungono a convergenza, e ogni grafo richiede lo stesso numero di iterazioni in ogni build, precisione e quota.

3.4 Misura dei tempi

Il tempo riportato per ogni esecuzione è quello del solo ciclo delle iterazioni: l'orologio si legge appena prima della prima iterazione e appena dopo l'ultima, chiamando `clock_gettime` con `CLOCK_MONOTONIC` (la funzione `wall_seconds` nel codice). Si ottiene così il tempo reale trascorso, come lo misurerebbe un cronometro, e non il tempo di CPU, che con più thread in parallelo sarebbe la somma del tempo di lavoro di ciascuno. Restano fuori dalla misura le operazioni fatte una volta sola: la lettura del file `.csr` e, nella versione ibrida, il trasferimento del grafo verso la GPU e quello dei rank finali verso l'host.

Per i tempi complessivi si è preferito un orologio dell'host agli eventi CUDA e al profiler, per due ragioni. I tempi delle quattro versioni vanno confrontati fra loro, e devono quindi venire dallo stesso orologio, ma la build sequenziale non ha né CUDA né OpenMP, mentre `clock_gettime` è disponibile ovunque. Inoltre gli eventi misurano solo ciò che accade negli stream della GPU, mentre nella versione ibrida una parte del lavoro la svolge la CPU.

Con la GPU sola (`-s 0`), il tempo misurato da `wall_seconds` differisce di al più 0,06 ms dall'intervallo che `nsys` registra fra la prima e l'ultima operazione della GPU.

Il profiler resta lo strumento per il dettaglio: `nsys` registra inizio e fine di ogni kernel e di ogni trasferimento, ed è da lì che vengono le durate delle singole fasi della versione ibrida.

3.5 Scalabilità della versione OpenMP

Ogni configurazione è stata eseguita tre volte; si riporta il minimo, visto che l'attività di sistema può rallentare un'esecuzione.

Con p thread, lo speed-up e l'efficienza parallela sono definiti come

$$S(p) = \frac{T_{\text{seq}}}{T(p)}, \qquad E(p) = \frac{S(p)}{p}, \tag{4}$$

dove T_{seq} è il tempo della build sequenziale (compilata senza `-fopenmp`) e $T(p)$ quello della build OpenMP a p thread.

Tabella 4: Tempi di esecuzione in secondi sulla Macchina 1 (4 core fisici, 8 thread). Lo speed-up dell'ultima colonna è calcolato tra la configurazione in doppia precisione più veloce e quella sequenziale.

Grafo	seq	1 thr	2 thr	4 thr	8 thr	float 8	speed-up
web-NotreDame	0,150	0,156	0,087	0,054	0,049	0,048	3,04×
web-Stanford	0,325	0,319	0,165	0,099	0,092	0,096	3,55×
web-Google	1,151	1,177	0,645	0,377	0,361	0,259	3,19×
web-BerkStan	0,658	0,665	0,355	0,216	0,192	0,171	3,43×
cit-Patents	1,199	1,225	0,729	0,481	0,466	0,284	2,57×
wiki-topcats	2,397	2,424	1,342	0,828	0,765	0,517	3,13×
soc-Pokec	4,098	4,115	2,293	1,401	1,400	0,907	2,93×
soc-LiveJournal1	11,198	11,461	6,575	4,449	4,229	3,234	2,65×

Nella Figura 5 le otto curve della Macchina 1 si appiattiscono fra 4 e 8 thread, e quasi tutte si allontanano dall'andamento lineare già a 2: fa eccezione `web-Stanford`, che a 2 thread ha ancora un'efficienza del 98 %. L'efficienza (Figura 6) è dell'82–98 % a 2 thread, del 62–82 % a 4 e del 32–44 % a 8.

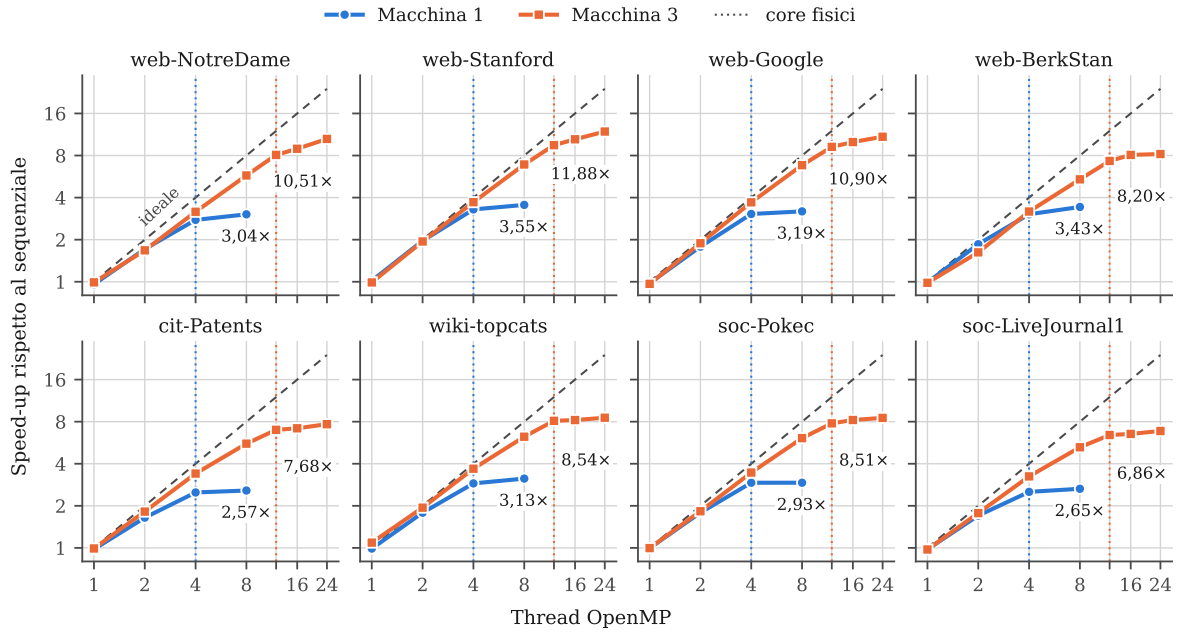


Figura 5: Scalabilità della versione OpenMP, un pannello per grafo e una curva per macchina, su assi logaritmici. Il riferimento per lo speed-up è la build sequenziale di ciascuna macchina. La linea tratteggiata diagonale indica lo speed-up lineare ideale $S(p) = p$, le linee continue quelli misurati, con il massimo raggiunto in etichetta; le linee punteggiate verticali segnano i core fisici di ciascuna macchina, 4 e 12. Figura generata da `tools/plot_results.py` a partire da `results/machine1/bench.csv` e `results/machine3/bench.csv`.

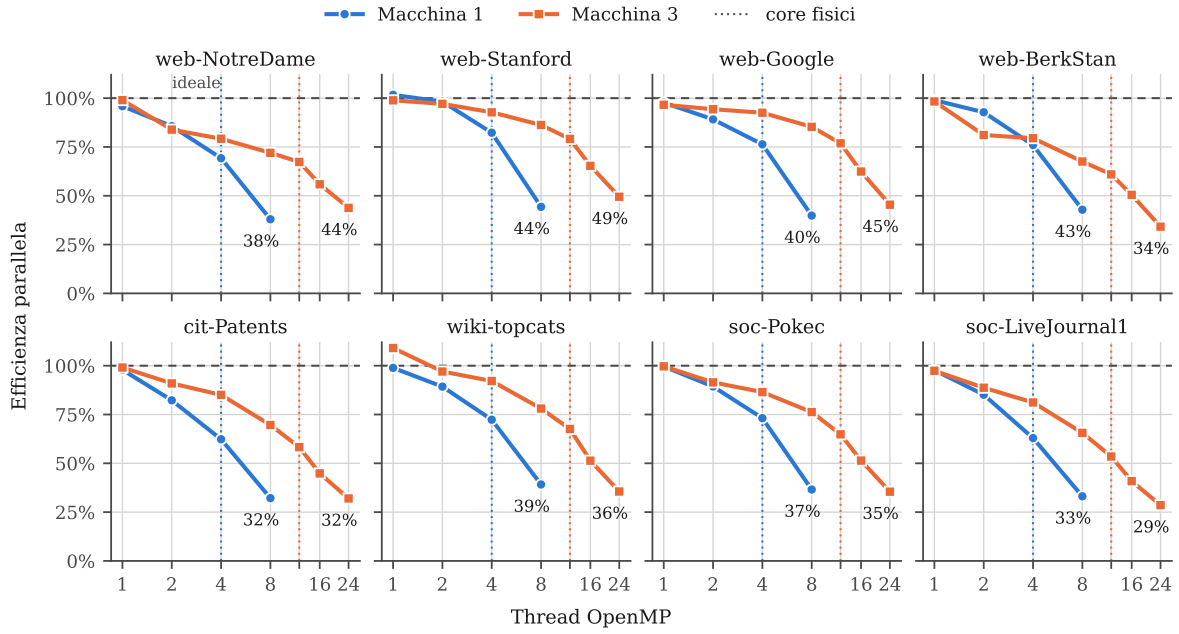


Figura 6: Efficienza parallela della versione OpenMP, un pannello per grafo e una curva per macchina. La linea tratteggiata indica l'efficienza ideale del 100 %, le linee continue quelle misurate, con in etichetta il valore al massimo numero di thread; le linee punteggiate verticali segnano i core fisici, come nella Figura 5. Figura generata da `tools/plot_results.py` a partire da `results/machine1/bench.csv` e `results/machine3/bench.csv`.

Tabella 5: Tempi di esecuzione in secondi sulla Macchina 3 (12 core fisici, 24 thread). Lo speed-up è calcolato come nella Tabella 4.

		thread								
Grafo	**seq**	**1**	**2**	**4**	**8**	**12**	**16**	**24**	**float 24**	**speed-up**
web-NotreDame	0,142	0,144	0,085	0,045	0,025	0,018	0,016	0,014	0,014	10,51×
web-Stanford	0,334	0,338	0,172	0,090	0,048	0,035	0,032	0,028	0,021	11,88×
web-Google	1,081	1,119	0,573	0,292	0,158	0,117	0,108	0,099	0,079	10,90×
web-BerkStan	0,683	0,695	0,421	0,215	0,126	0,093	0,085	0,083	0,063	8,20×
cit-Patents	0,979	0,988	0,538	0,288	0,176	0,140	0,137	0,128	0,096	7,68×
wiki-topcats	1,936	1,775	0,998	0,525	0,310	0,239	0,236	0,227	0,183	8,54×
soc-Pokec	2,728	2,737	1,491	0,789	0,447	0,351	0,332	0,320	0,262	8,51×
soc-LiveJournal1	8,081	8,296	4,554	2,489	1,540	1,259	1,236	1,178	0,810	6,86×

Il calcolo è memory-bound. Passando da 4 a 8 thread il tempo, anziché dimezzarsi, scende al più dell'11 % (`web-BerkStan`), e su `soc-Pokec` resta invariato. Il processore della Macchina 1 possiede 4 core fisici presentati come 8 thread logici; i due thread che stanno sullo stesso core condividono anche cache e unità di accesso alla memoria. Poiché ogni thread passa la maggior parte del tempo in attesa della lettura `contrib[col_idx[j]]`, i thread aggiuntivi raddoppiano le richieste, non le unità che le servono. Il *simultaneous multithreading* (SMT) rende quando un thread calcola mentre l'altro aspetta, ma qui aspettano entrambi.

Con più core fisici. Se il limite sono le unità di accesso alla memoria di ciascun core, a spostarlo deve essere il numero di core fisici, non quello dei thread. La Macchina 3, con 12 core fisici e 24 thread logici, lo conferma (Tabella 5 e Figura 5). Finché ogni thread nuovo trova un core libero lo speed-up continua a crescere: a 8 thread l'efficienza è ancora del 66–86 %, contro il 32–44 % della Macchina 1, e a 12 del 54–79 %, poco sotto il 62–82 % che la Macchina 1 ha a 4, quando anche lì ogni core ha il suo thread. Oltre i 12 thread osserviamo lo stesso fenomeno della Macchina 1, con lo speed-up che cresce più lentamente e l'efficienza che cala più rapidamente. La curva si piega fortemente, dunque, in corrispondenza della fine dei core fisici.

Anche per la Macchina 3 la resa di SMT segue la memoria. Il guadagno da 12 a 24 thread è massimo nei grafi minori, come `web-NotreDame` e `web-Stanford`, che sono gli unici due grafi i cui dati — il CSR e i tre vettori `cur`, `nxt` e `contrib` — stanno per intero nei 21 MiB della L3.[3] In sostanza, il thread gemello rende quando l'attesa è breve, come quando i dati arrivano dalla cache, e poco quando arrivano dalla memoria.

`web-Google` impiega più tempo di `web-BerkStan`. A 4 thread e sulla Macchina 1 `web-Google` impiega 0,377 s contro i 0,216 s di `web-BerkStan`, il 74 % in più, e non è una questione di iterazioni perché ne esegue di meno (62 contro 64). Non è nemmeno una questione di lavoro svolto, perché `web-BerkStan` ha il 49 % di archi in più: a parità di arco `web-Google` costa 1,19 ns [4] contro 0,44 ns di `web-BerkStan`, più di due volte e mezzo tanto.

Ogni riga — i predecessori di un nodo — porta un costo fisso (leggere i due estremi in `row_ptr`, scrivere il rank nuovo e aggiornare l'errore), e `web-Google` ha il 28 % di nodi in più. Quel 28 % è però un tetto di miglioramento ideale: lo si raggiungerebbe solo se gli archi fossero gratis, mentre il divario osservato è il 74 %. Non lo spiega neppure la concentrazione nelle righe lunghe:

[3] `web-NotreDame`: 9,4 MiB di CSR più tre vettori da 2,5 MiB, 16,9 MiB in tutto; `web-Stanford`: 12,0 MiB più tre vettori da 2,2 MiB, circa 18,5 MiB.

[4] 0,377 s / 62 iterazioni = 0,00608 s; 0,00608 s / 5105039 archi = 1,19 ns per arco.

`soc-LiveJournal1` ha il 77 % degli archi oltre i 16 predecessori, più del 74 % di `web-BerkStan` (Tabella 1), eppure per arco costa 1,26 ns, poco più di `web-Google`.

La differenza sta in quanto distano un indice e il successivo, visto che il ciclo recupera i contributi dei predecessori `contrib[col_idx[j]]` in ordine sparso, perché i valori di `col_idx` non sono sequenziali. In `web-BerkStan` i predecessori di un nodo hanno indici quasi consecutivi, infatti il salto mediano fra due indici della stessa riga è 1, una conseguenza di come sono stati raccolti e salvati i dati: una sola linea di cache da 64 byte porta con sé 8 contributi consecutivi, e ne bastano poche per servire un'intera riga: in media una linea ogni 16 archi, 0,06 linee per arco (Tabella 6), cosicché questa raccolta di valori da posizioni sparse si riduce a una lettura quasi sequenziale. In `web-Google` gli indici sono sparsi su tutto l'array, con un salto mediano di quasi 17 000 posizioni, e quasi ogni arco pretende una linea di cache nuova (0,99 linee per arco)[5]: 64 byte trasferiti per usarne solo 8. Su una iterazione fanno 309 MiB di linee di cache richieste contro 29. Le letture sequenziali vanno nel verso opposto: `col_idx`, `row_ptr` e i vettori di rank (`nxt` e `cur`) fanno 53 MiB su `web-BerkStan` contro 50 su `web-Google`. La Tabella 7 mostra la riga del nodo 1 nei due grafi.

Le linee richieste non costano tutte uguale: `contrib` occupa 6,7 MiB su `web-Google` e 5,2 MiB su `web-BerkStan`, abbastanza poco perché una parte degli accessi sparsi resti servita dalla cache invece di arrivare in memoria. Gli altri due grafi web, `web-Stanford` e `web-NotreDame`, lo mostrano dal lato opposto: `web-Stanford` ha indici sparsi quasi quanto `web-Google`, 0,91 linee di cache per arco contro 0,99, ma il suo `contrib` di 2,2 MiB sta per intero in cache con ampio margine, e il divario con `web-NotreDame`, locale come `web-BerkStan`, si riduce a 0,68 contro 0,62 ns per arco. Che `web-NotreDame` costi per arco più di `web-BerkStan` dipende invece dalla lunghezza delle righe, 4,6 predecessori in media contro 11,1: il costo fisso di ogni riga si divide su meno archi.

La cache manca del tutto alle quattro istanze maggiori, i cui `contrib`, da 12,5 a 37 MiB, nella L3 da 8 MiB non entrano: la gran parte delle linee che la lettura sparsa richiede va cercata in memoria, e il costo per arco dipende sia dalla località sia da quanta parte di `contrib` la cache riesce ancora a trattenere. `wiki-topcats`, la più locale delle quattro (0,42 linee per arco), costa 0,79 ns per arco; `cit-Patents`, la meno locale (0,96) e con le righe più corte, 1,94 ns, il valore più alto di tutti. `soc-Pokec` ha indici più sparsi di `soc-LiveJournal1` (0,80 linee per arco contro 0,52) ma un `contrib` tre volte più piccolo, e costa meno (1,04 contro 1,26 ns); `soc-LiveJournal1`, a sua volta, nonostante una località migliore di quella di `web-Google`, paga per arco un po' di più.

La Macchina 3 conferma questa lettura dal lato dell'hardware (colonne M1 e M3 della Tabella 6). Core per core non è più veloce: da sequenziale, sui quattro grafi web, le due CPU si equivalgono entro il 6 %, due volte a favore dell'una e due dell'altra. Ha però una L3 di 21 MiB anziché 8, e a 4 thread il suo vantaggio è modesto sui quattro grafi web, il cui `contrib` sta già nella L3 della Macchina 1, e forte sulle quattro istanze maggiori, il cui `contrib` non ci sta. Ciò che la Macchina 3 ha in più sta dunque nella gerarchia di memoria, e rende proprio dove la Macchina 1 aspetta la RAM.

3.6 Confronto fra le varianti `float` e `double`

Sulla Macchina 1, a 8 thread e a parità di numero di iterazioni, la variante in singola precisione riduce il tempo fra il 24 e il 39 % sulle cinque istanze in cui `contrib`, in doppia precisione, occupa quasi tutta la L3 o la supera: `web-Google`, `cit-Patents`, `wiki-topcats`, `soc-Pokec` e `soc-LiveJournal1`. Sugli altri tre grafi web, dove `contrib` sta in cache o viene letto quasi

[5] Linee di cache distinte contate da `tools/locality_stats.py`; 5054775 linee / 5105039 archi = 0,99 linee per arco; 5054775 linee × 64 byte = 309 MiB per iterazione.

Tabella 6: Località della lettura sparsa su `contrib`. Il salto è la mediana di `col_idx[j+1] - col_idx[j]` all'interno di una riga; le linee di cache distinte sono contate su chunk di 256 righe, l'unità con cui il ciclo distribuisce le righe ai thread, e divise per il numero di archi danno le linee per arco; le letture per iterazione sono il volume che ne risulta. Misure prodotte da `tools/locality_stats.py` dai file `.csr`; i tempi per arco vengono dai 4 thread della Tabella 4 per la Macchina 1 (M1) e della Tabella 5 per la Macchina 3 (M3).

Grafo	**salto**	**linee**	**letture**	`contrib`	**ns per arco**	
	mediano	**per arco**	**per iter.**		**M1**	**M3**
web-NotreDame	1	0,07	7 MiB	2,5 MiB	0,62	0,52
web-Stanford	1 080	0,91	129 MiB	2,2 MiB	0,68	0,62
web-Google	16 836	0,99	309 MiB	6,7 MiB	1,19	0,92
web-BerkStan	1	0,06	29 MiB	5,2 MiB	0,44	0,44
cit-Patents	60 903	0,96	973 MiB	28,8 MiB	1,94	1,16
wiki-topcats	59	0,42	725 MiB	13,7 MiB	0,79	0,50
soc-Pokec	10 319	0,80	1 501 MiB	12,5 MiB	1,04	0,59
soc-LiveJournal1	3 104	0,52	2 172 MiB	37,0 MiB	1,26	0,71

Tabella 7: I primi cinque predecessori del nodo 1 come finiscono in `col_idx`, con la distanza fra un indice e il successivo.

Grafo	**Primi cinque indici in `col_idx`**				
`web-Google`	193 140	262 338	525 606	541 488	713 368
salti		69 198	263 268	15 882	171 880
`web-BerkStan`	2	6	15	17	19
salti		4	9	2	2

in sequenza, il guadagno scende all'11 % su `web-BerkStan` e sparisce sugli altri due, dove la differenza (+3 % su `web-NotreDame`, −4 % su `web-Stanford`) può essere attribuita al rumore di misura. Il guadagno si concentra dunque dove la lettura sparsa va in memoria, ed è coerente con il dimezzamento dei byte trasferiti per ogni rank: un'ulteriore conferma indiretta della natura memory-bound del calcolo.

Sulla Macchina 3, a 24 thread, il guadagno si allarga a sette grafi su otto, dal 18 al 31 %, e comprende `web-Stanford` e `web-BerkStan`, che sulla Macchina 1 guadagnavano poco o nulla. Resta nullo solo su `web-NotreDame` (−2 %), il più piccolo degli otto grafi e quello con le letture per iterazione più leggere.

Il limite della singola precisione si manifesta sulla soglia di convergenza: su `web-Google` l'errore L_1 non scende sotto $\approx 5 \cdot 10^{-9}$, per cui una tolleranza di 10^{-9} non viene mai raggiunta, mentre la doppia precisione la raggiunge in 104 iterazioni. Alla tolleranza utilizzata negli esperimenti (10^{-6}) entrambe convergono nello stesso numero di iterazioni.

3.7 Risultati della versione ibrida

La versione ibrida è stata misurata sulle tre macchine e su ciascuna delle loro GPU, sugli stessi nove grafi, in doppia e in singola precisione, alle quote 0, 0,05, 0,1, 0,25, 0,5, 0,75 e 1. La parte della CPU usa tutti i thread logici: 8 sulle Macchine 1 e 2, 24 sulla Macchina 3. Come per OpenMP, ogni configurazione è stata eseguita tre volte e si riporta il minimo. Ogni esecuzione della versione ibrida è avvenuta sotto il profiler `nsys`, che introduce un overhead di circa l'1 % del tempo e registra inizio e fine di ogni kernel e di ogni copia (i report di `nsys` sono letti

da `tools/nsys_phases.py`). Tutti i tempi di questa sezione sono della variante in doppia precisione, tranne quelli del paragrafo sulla singola precisione.

La GPU da sola. Con il lavoro tutto sulla GPU (Tabella 8) la RTX 2080 Ti è da 1,4 a 2,9 volte più veloce dei 24 thread della Macchina 3, e da 15 a 27 volte più veloce della versione sequenziale. Sulla Macchina 1, invece, la GTX 1050 Ti e i 4 core della CPU si equivalgono: la GPU va da 0,68 volte (`web-Google`) a 1,9 volte (`web-BerkStan`) la versione OpenMP a 8 thread. Il trasferimento iniziale del grafo, escluso dal tempo, non cambia il quadro: per `soc-LiveJournal1`, 392 MB, richiede da 37 a 63 ms secondo la macchina, contro 0,47 s di calcolo sulla RTX 2080 Ti.

Tabella 8: Tempi in secondi della GPU sola (quota 0) e, per confronto, della migliore configurazione OpenMP della stessa macchina (8 thread sulla Macchina 1, 24 sulla Macchina 3). La RTX 2080 Ti è quella della Macchina 3 sul collegamento x16: con la GPU sola, sull'altra RTX 2080 Ti della Macchina 3 e su quella della Macchina 2 i tempi coincidono entro il 3 %. Sulla Macchina 2 la versione OpenMP non è stata misurata.

	Macchina 1		**Macchina 2**	**Macchina 3**	
Grafo	**GTX 1050 Ti**	**OpenMP**	**GTX 1080 Ti**	**RTX 2080 Ti**	**OpenMP**
web-NotreDame	0,032	0,049	0,010	0,010	0,014
web-Stanford	0,127	0,092	0,019	0,014	0,028
web-Google	0,534	0,361	0,113	0,041	0,099
web-BerkStan	0,101	0,192	0,027	0,028	0,083
cit-Patents	0,444	0,466	0,110	0,049	0,128
wiki-topcats	0,777	0,765	0,188	0,085	0,227
soc-Pokec	1,965	1,400	0,435	0,161	0,320
soc-LiveJournal1	3,922	4,229	1,023	0,466	1,178

Anche sulla GPU il costo per arco dipende dalla località. Sulla GTX 1050 Ti un arco di `web-Google` costa 1,69 ns e uno di `web-BerkStan` 0,21 ns, otto volte meno; sulla GTX 1080 Ti il rapporto è di 6,4, sulla RTX 2080 Ti di 2,2. Su tutte e tre le GPU la memoria globale si legge in settori da 32 byte: un arco che chiede un contributo lontano dagli altri ne impegna uno intero per 8 byte utili; sulla CPU, per gli stessi 8 byte, impegna una linea di cache da 64 byte.

Le classi contro un thread per riga. Per misurare che cosa rendono le tre classi del secondo ciclo, la GPU sola è stata eseguita anche con il kernel ingenuo, un thread per riga, che si ottiene con `-b 1000000,1000000`: una soglia oltre la riga più lunga dà a ogni riga il suo thread. Le due versioni sono state misurate nella stessa sessione, con `tools/granularity.sh`, sulla GTX 1050 Ti della Macchina 1, sulla GTX 1080 Ti della Macchina 2 e sulla RTX 2080 Ti della Macchina 3, a x16 (Tabella 9); la RTX 2080 Ti della Macchina 2, che con la GPU sola dà gli stessi tempi di quella della Macchina 3, non è stata misurata. Secondo `nsys` la differenza fra le due versioni è quasi tutta nell'accumulo: `contrib_kernel` cambia al più di qualche centesimo di millisecondo.

I grafi si dividono in tre gruppi. Nel primo, `web-NotreDame`, `web-Stanford`, `web-BerkStan` e `wiki-topcats`, le classi sono da 2,9 a 19 volte più veloci. Su questi grafi la riga più lunga è enorme rispetto al resto del lavoro, fra lo 0,7 e l'1,7 % degli archi. Nel kernel ingenuo la somma un solo thread, un predecessore dopo l'altro, e il kernel finisce solo quando quel thread ha finito; con le classi se la dividono i 256 thread di un blocco: su `wiki-topcats` sono 238 040 predecessori per un solo thread, contro 930 per ciascuno dei 256.

Tabella 9: Tempi in secondi della GPU sola con il kernel ingenuo, un thread per riga, e con le classi 16 e 256; × è quante volte le classi sono più veloci. Riga max è la lunghezza della riga più lunga, in percentuale degli archi. Le classi sono state rimisurate insieme all'ingenuo. La RTX 2080 Ti è quella della Macchina 3, a x16.

		GTX 1050 Ti			**GTX 1080 Ti**			**RTX 2080 Ti**		
Grafo	**riga max**	**ingenuo**	**classi**	×	**ingenuo**	**classi**	×	**ingenuo**	**classi**	×
web-NotreDame	0,72	0,086	0,027	3,19	0,061	0,010	6,28	0,046	0,010	4,80
web-Stanford	1,67	0,580	0,126	4,59	0,230	0,019	11,87	0,152	0,014	10,87
web-Google	0,12	0,608	0,534	1,14	0,172	0,113	1,51	0,062	0,041	1,53
web-BerkStan	1,11	0,677	0,094	7,24	0,516	0,027	19,13	0,342	0,028	12,10
cit-Patents	0,005	0,439	0,444	0,99	0,112	0,110	1,02	0,050	0,049	1,02
wiki-topcats	0,83	2,265	0,777	2,91	1,393	0,188	7,40	0,627	0,085	7,35
soc-Pokec	0,045	2,302	1,963	1,17	0,629	0,435	1,45	0,214	0,160	1,34
soc-LiveJournal1	0,020	4,873	3,921	1,24	1,502	1,023	1,47	0,566	0,465	1,22

Nel secondo gruppo, `web-Google`, `soc-Pokec` e `soc-LiveJournal1`, le classi sono da 1,1 a 1,5 volte più veloci. Qui la riga più lunga pesa al più lo 0,12 % degli archi e finisce prima del resto. Questi grafi hanno invece molte righe medie, da 17 a 256 predecessori, che portano dal 44 all'81 % degli archi (Tabella 1): il guadagno viene con ogni probabilità da queste righe, che con le classi un warp si divide invece di lasciarle a un thread.

Il terzo gruppo è `cit-Patents`, dove le due versioni sono alla pari entro il 2 %. La sua riga più lunga ha appena 779 predecessori, e tre quarti degli archi stanno in righe fino a 16, che le classi lasciano comunque a un thread ciascuna, come l'ingenuo: senza hub le classi non servono, ma nemmeno costano.

La singola precisione sulla GPU. La variante `float` dimezza i byte dei tre vettori, ma sulla GPU non sempre conviene. Sulla RTX 2080 Ti riduce il tempo della GPU sola del 27 % su `soc-LiveJournal1` e del 38 % su `cit-Patents`, ma lo aumenta del 22–31 % sui tre grafi dove la lettura di `contrib` è più leggera: `web-NotreDame` e `web-BerkStan`, i più locali, e `web-Stanford`, il cui `contrib`, di 2,2 MiB, è il più piccolo fra gli otto grafi misurati. Sulla GTX 1050 Ti rallentano solo `web-NotreDame` e `web-BerkStan`, dell'11 e del 13 %; sulla GTX 1080 Ti nessun grafo. Secondo `nsys` il rallentamento è tutto nei kernel di accumulo (`accumulate_thread_rows`, `accumulate_warp_rows` e `accumulate_block_rows`), mentre `contrib_kernel` in `float` dura meno della metà. La spiegazione più probabile sta negli accumulatori, che restano in `double` (Sezione 2.4): nella variante `float` ogni contributo letto va convertito in `double` prima di essere sommato, un'istruzione in più per arco, eseguita dalle unità a 64 bit, che sulle GPU di fascia consumer sono una piccola frazione di quelle a 32. I due effetti del `float` vanno in direzioni opposte. Su `cit-Patents` e `soc-LiveJournal1` i contributi sono sparsi su tutto il vettore, e la GPU passa il tempo ad aspettarli dalla memoria: leggerne la metà dei byte accorcia l'attesa, e il `float` è più veloce. Su `web-NotreDame`, `web-BerkStan` e `web-Stanford` i contributi sono vicini fra loro o stanno in cache e arrivano in fretta: la GPU non aspetta la memoria, e la conversione in più per ogni arco allunga il tempo.

Dividere il lavoro. La Tabella 10 riporta per ogni configurazione la quota più veloce, e la Figura 7 l'andamento completo per quattro di esse. Il quadro si divide in due. Sulla Macchina 1 dividere il lavoro conviene su tutti i grafi: la quota migliore è fra 0,5 e 0,75 su sette grafi su otto, e la versione ibrida è da 1,05 a 1,80 volte più veloce della migliore fra OpenMP e GPU sola, da 4,2 a 7,3 volte più veloce della sequenziale, contro il 2,6–3,5 della sola OpenMP. Sulle GPU più veloci, invece, la GPU sola è quasi sempre la scelta migliore: una quota rende sui due

grafi sociali, i più grandi per numero di archi, e al più 1,25 volte (`soc-Pokec` sulla GTX 1080 Ti), 1,20 sulla RTX 2080 Ti a x16, 1,08 a x8, mai sulla RTX 2080 Ti della Macchina 2, dove la CPU ha 4 core, come quella della Macchina 1, invece dei 12 della Macchina 3: gli stessi core che bastano accanto alla GTX 1050 Ti sono troppo lenti accanto alla RTX 2080 Ti. La GTX 1080 Ti guadagna qualcosa, con quote piccole, anche su `web-Google` e `wiki-topcats`.

Tabella 10: La quota più veloce per ogni grafo e configurazione, e fra parentesi lo speed-up che dà rispetto alla GPU sola. Un trattino dove nessuna quota batte la GPU sola di più dell'1 %.

	GTX 1050 Ti	**GTX 1080 Ti**	**RTX 2080 Ti**		
Grafo	**M1**	**M2**	**M2**	**M3, x8**	**M3, x16**
web-NotreDame	0,5 (1,05)	—	—	—	—
web-Stanford	0,75 (1,66)	—	—	—	—
web-Google	0,75 (1,99)	0,1 (1,05)	—	—	—
web-BerkStan	0,1 (1,12)	—	—	—	—
cit-Patents	0,5 (1,70)	—	—	—	—
wiki-topcats	0,5 (1,82)	0,05 (1,02)	—	—	—
soc-Pokec	0,75 (2,04)	0,25 (1,25)	—	0,1 (1,08)	0,25 (1,17)
soc-LiveJournal1	0,5 (1,70)	0,25 (1,14)	—	0,1 (1,03)	0,25 (1,20)

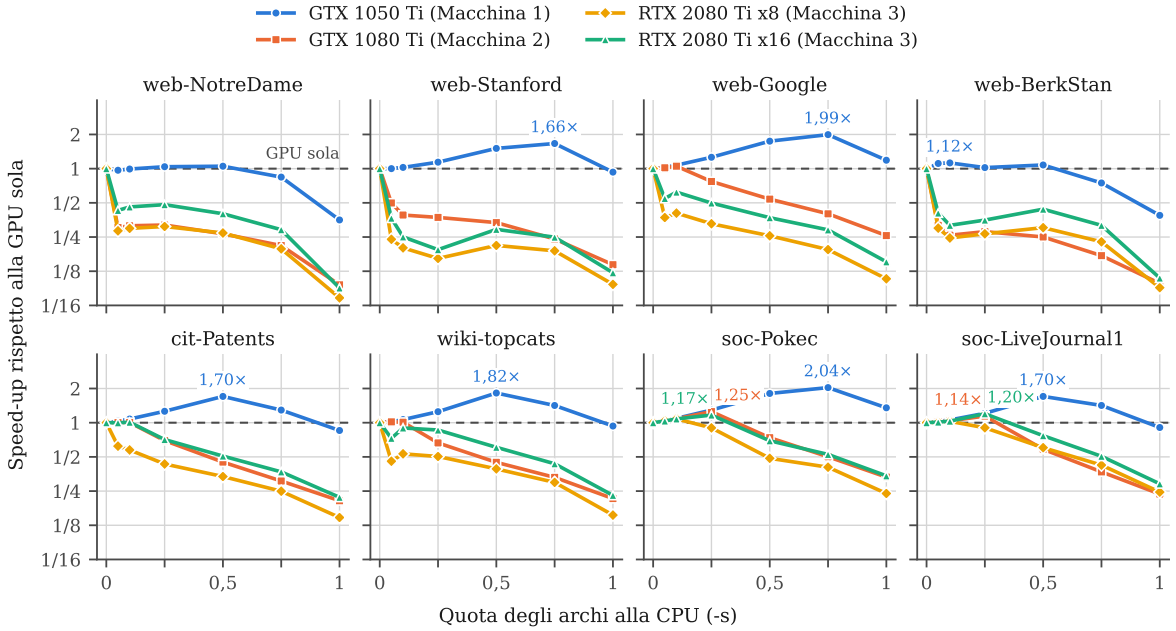


Figura 7: Speed-up della versione ibrida rispetto alla GPU sola, al variare della quota di archi data alla CPU, un pannello per grafo, su asse verticale logaritmico. Sopra la linea tratteggiata dividere il lavoro conviene. Figura generata da `tools/plot_hybrid.py`.

Dove va il tempo di un'iterazione. La Tabella 11 scompone un'iterazione di `soc-LiveJournal1` con i tempi di `nsys`, e mostra perché dividere il lavoro conviene sulla Macchina 1 e quasi mai sulle GPU più veloci. Dopo `contrib_kernel` il lavoro procede su due strade parallele (Figura 4): la GPU fa l'accumulo delle sue righe; la CPU attende la copia di `contrib`, fa l'accumulo delle sue e rimanda i rank. L'iterazione dura quanto la più lunga delle due.[6] Sulla Macchina 1 a quota 0,5 la strada della GPU è la più lunga, 43,5 ms

[6] iter. $\approx$ contrib + max(accumulo GPU, copia + accumulo CPU) + ritorno.

di accumulo contro i 3,6 + 30,4 ms della CPU, che la attende per 9,9 ms; a 0,75 accade il contrario, e la strada più lunga è quella della CPU. La quota migliore, quella in cui le due strade durano uguale, sta quindi fra le due, poco sopra 0,5. Sulla RTX 2080 Ti a x16, a quota 0,25, le due strade quasi si equivalgono, 7,3 ms la GPU e 6,7 ms la CPU, e l'iterazione scende da 9,1 a 7,6 ms. Sullo stesso grafo e alla stessa quota, a x8 la sola copia costa 5,7 ms invece di 2,9 ms: la strada della CPU arriva a 9,5 ms, più dei 7,3 ms della GPU e più anche dei 9,1 ms dell'intera iterazione con la GPU sola. Sulla Macchina 2, con 4 core invece di 12, la strada della CPU dura 5,9 + 11,1 ms: tiene il passo della GTX 1080 Ti, il cui accumulo dura 16,5 ms, ma non della RTX 2080 Ti, che impiega 7,2 ms.

Tabella 11: Le fasi di un'iterazione di `soc-LiveJournal1`, in millisecondi, misurate con `nsys`. La copia è quella di `contrib` verso l'host, il ritorno quello dei rank della CPU più `scatter_host_ranks`, l'attesa quella dell'host per la GPU a fine iterazione. La colonna iter. è la durata dell'intera iterazione, contrib quella di `contrib_kernel`. Ogni riga viene dalla più veloce delle tre esecuzioni della sua configurazione. Ogni valore è la mediana della durata di quella fase sulle iterazioni di quell'esecuzione, calcolata colonna per colonna: i valori di una stessa riga possono quindi venire da iterazioni diverse, e ricompongono iter. con una variazione di 0,1–0,2 ms. Con la GPU sola le fasi della CPU mancano, e sulla Macchina 3 i tempi a x8 e a x16 coincidono, quindi la tabella ne riporta una riga sola.

GPU		quota	iter.	contrib	accumulo GPU	copia	accumulo CPU	ritorno	attesa
GTX 1050 Ti	M1	0	76,9	1,2	75,6	—	—	—	—
		0,5	45,1	1,2	43,5	3,6	30,4	0,4	9,9
		0,75	54,4	1,2	23,5	3,6	48,6	0,9	0,9
GTX 1080 Ti	M2	0	20,1	0,3	19,7	—	—	—	—
		0,25	17,5	0,3	16,5	5,9	11,1	0,1	0,1
RTX 2080 Ti	M2	0,25	17,4	0,2	7,2	5,9	11,1	0,1	0,1
	M3	0	9,1	0,2	8,9	—	—	—	—
	M3 x8	0,25	10,0	0,2	7,3	5,7	3,8	0,1	0,1
	M3 x16	0,05	9,0	0,2	8,7	2,9	0,8	0,0	5,0
		0,25	7,6	0,3	7,3	2,9	3,8	0,1	0,6

Gli hub costano poco alla GPU. Le quote piccole, che danno alla CPU i soli hub, non rendono quasi mai, e la ragione sta in un'ipotesi che le misure smentiscono: gli hub sono le righe che la GPU calcola meglio. Su `soc-LiveJournal1`, con la RTX 2080 Ti a x16, la quota 0,05 toglie alla GPU il 5 % degli archi, ma il suo accumulo scende solo da 8,86 a 8,73 ms, l'1,5 %: quegli archi le costavano 0,04 ns l'uno, contro una media di 0,13 ns. Il kernel a blocchi li legge con accessi coalescenti, e su righe così lunghe i thread del blocco restano tutti al lavoro. La copia di `contrib`, invece, è il costo fisso della versione ibrida, $8N$ byte per iterazione qualunque sia la quota, e per guadagnare la CPU deve togliere alla GPU abbastanza lavoro da ripagarlo. Sulla GTX 1050 Ti, dove ogni arco costa alla GPU molto di più, basta poco; sulle GPU più veloci servono quote alte, che però allungano la strada della CPU oltre quella della GPU.

Il collegamento. La banda della copia di `contrib`, misurata da `nsys`, è di 11,2 GB/s sulla Macchina 1, a x16, di 6,6 GB/s sulle due GPU della Macchina 2, entrambe a x8, e di 6,8 GB/s e 13,5 GB/s sulle due RTX 2080 Ti della Macchina 3, a x8 e a x16. Con la GPU sola il collegamento non conta: le due schede della Macchina 3 danno tempi uguali entro il 3 %, perché durante il calcolo sul bus non passa quasi nulla. Con una quota per la CPU, invece, è il collegamento a decidere: a parità di GPU e di CPU, la x16 rende la versione ibrida 1,20 volte più veloce della GPU sola su `soc-LiveJournal1`, la x8 appena 1,03.

Pochi thread sulle righe della CPU. Su `web-Stanford` e `web-BerkStan` l'accumulo della CPU non cresce in modo regolare con la quota. Sulla RTX 2080 Ti della Macchina 3, a x16, quello di `web-Stanford` dura per iterazione 0,33 ms a quota 0,05, 0,61 ms a 0,1, 0,82 ms a 0,25 e solo 0,48 ms a 0,5, pur con il doppio degli archi rispetto a 0,25. L'accumulo della CPU dà le righe ai thread in chunk di 16, `schedule(dynamic, 16)`, e su questi grafi le righe della CPU sono poche e lunghissime. Su `web-Stanford`, a quota 0,05 e 0,1 sono 4 e 10, un solo chunk: lavora un solo thread su 24. A 0,25 sono 92, sei chunk, e lavorano sei thread. A 0,5 sono 2 519, e lavorano tutti i 24 thread. Conta quindi quanti thread si dividono il lavoro, più che quanti archi ci sono.

4 Valutazioni e osservazioni

4.1 Punti di forza

- **Assenza di sincronizzazione nel nucleo di calcolo.** La formulazione *pull*, con il grafo memorizzato trasposto, elimina alla radice le scritture concorrenti, e si trasferisce invariata alla versione CUDA.
- **Il CSR è adattato al problema.** Il valore di un non nullo dipende solo dalla colonna, non dal singolo arco: il formato generico obbligherebbe a ripetere lo stesso $1/\text{outdeg}(u)$ una volta per arco uscente di u, mentre `out_deg[N]` lo registra una volta sola. È un'informazione duplicata che non viene scritta.
- **Il vettore `contrib` riduce gli accessi sparsi.** Senza di esso il ciclo di accumulo dovrebbe leggere, per ogni arco, `cur[u]` e `out_deg[u]`: due array distinti, quindi due accessi imprevedibili invece di uno. Precalcolare il quoziente porta le divisioni da M a N, ma soprattutto lascia una lettura sola per arco.
- **Verifica indipendente e collaudata.** Tre implementazioni concordano, e lo strumento di verifica è stato validato anche con un'iniezione di guasto.
- **Un solo codice per la GPU sola e per l'ibrida.** Con quota 0 la versione ibrida è la versione solo GPU, senza copie e senza thread OpenMP: le due non possono divergere, e il confronto fra loro misura soltanto l'effetto della quota.
- **Portabilità verificata.** Lo stesso sorgente, ricompilato su ogni macchina, è stato eseguito senza modifiche su cinque GPU, di tre modelli e due architetture, con tre versioni del CUDA Toolkit; griglie e memoria sono dimensionate a tempo di esecuzione.
- **Granularità adattiva.** Nella versione solo GPU le tre classi del secondo ciclo confrontate con il kernel ingenuo, un thread per riga, nel caso peggiore pareggiano e nel migliore rendono la GPU 19 volte più veloce.

4.2 Punti deboli e limiti

- **Il convertitore non scala oltre le istanze usate.** Il convertitore Python mantiene l'intera edge list in memoria: se in futuro si volessero trattare grafi sensibilmente più grandi, andrebbe ripensato.
- **Formato binario non standard.** Il formato `.csr` è specifico del progetto: è velocissimo da caricare e tiene il programma C libero da un parser, ma non è ispezionabile con strumenti esterni.

- **La copia di `contrib` a ogni iterazione.** Con qualunque quota diversa da 0, l'intero vettore `contrib` passa sul bus a ogni iterazione, $8N$ byte in doppia precisione: è il costo che decide la convenienza della versione ibrida (Sezione 3.7).
- **Thread inattivi sulle righe della CPU.** Nella versione ibrida l'accumulo della CPU usa `schedule(dynamic, 16)`: le righe vanno ai thread in chunk di 16, e quando sono poche e lunghe finiscono in pochi chunk, e lavorano pochi thread, a volte uno solo, mentre gli altri restano fermi (Sezione 3.7).

4.3 Possibili miglioramenti

- **Ricerca su politica di scheduling e dimensione del chunk.** Nella versione OpenMP il chunk di `schedule(dynamic, 256)`, fissato a 256 senza una ricerca sistematica, non governa solo il bilanciamento del carico: chunk più ampi danno a un thread più righe consecutive, quindi più riuso delle linee già in cache. L'ottimo è un compromesso fra le due cose e dipende dal grafo; varrebbe la pena cercarlo, confrontando anche politiche come `guided`.
- **Soglie della granularità adattiva.** Nella versione ibrida la GPU dà un thread alle righe fino a 16 predecessori, un warp a quelle fino a 256 e un blocco alle più lunghe (opzione `-b`). Queste due soglie sono state fissate dopo poche prove, senza un'analisi approfondita, e quelle migliori dipendono verosimilmente dal grafo e dalla GPU: andrebbero cercate in modo sistematico, misurando i tempi di accumulo al variare delle due soglie. Il confronto con il kernel ingenuo, un thread per riga, mostra che le classi rendono, ma non quali soglie rendano di più.
- **Bilanciamento automatico della quota.** Nella versione ibrida la quota di archi assegnata alla CPU è un parametro (`-s`), e la migliore è quella con cui CPU e GPU finiscono il proprio lavoro insieme: dipende quindi dal grafo e dal rapporto fra la velocità della CPU e quella della GPU della macchina. Il programma potrebbe trovarla da solo nelle prime iterazioni: se a fine iterazione la CPU ha aspettato la GPU, le cede altre righe; se invece è la GPU ad aver aspettato, gliene toglie.
- **Copiare solo i contributi che servono alla CPU.** Nella versione ibrida, le righe della CPU non leggono tutto `contrib`: su `soc-LiveJournal1` a quota 0,25 i loro predecessori distinti sono il 36 % dei nodi. Un kernel che raccogliesse quei soli contributi in un vettore compatto ridurrebbe la copia a circa un terzo: sulla RTX 2080 Ti della Macchina 3, a x8, da 5,7 a circa 2 ms, abbastanza perché anche lì la quota 0,25 batta la GPU sola. Il guadagno dipende dal grafo: su `wiki-topcats`, alla stessa quota, i predecessori distinti sono l'89 % dei nodi.
- **Righe della CPU una alla volta.** Con `schedule(dynamic, 1)` nell'accumulo della CPU della versione ibrida ogni thread prenderebbe una riga per volta: su righe così lunghe il costo della distribuzione è trascurabile, e nessun thread resterebbe inattivo.
- **Accumulare in singola precisione nella variante `float`.** Se il rallentamento della singola precisione sulla GPU viene dalla conversione dei contributi in `double`, accumulare in `float` lo eliminerebbe, a prezzo dell'accuratezza, da verificare sulla soglia di convergenza.

4.4 Valutazione complessiva

La convenienza della versione ibrida dipende dal rapporto fra tre velocità: quella della GPU, quella della CPU e quella del collegamento fra le due. Dove GPU e CPU si equivalgono, come

sulla Macchina 1, dividere il lavoro rende fino a 1,8 volte la migliore delle due da sola; dove la GPU è molto più veloce, la copia a ogni iterazione costa più di quanto la CPU possa restituire, e conviene lasciarle tutto.

Sulla GPU conta anche come il lavoro è diviso fra i thread: dare alle righe lunghe un warp o un blocco invece di un thread solo la rende fino a 19 volte più veloce.

Il filo comune a tutte le misure è che il calcolo è limitato dalla memoria, sulla CPU come sulla GPU. I tempi li spiegano la località delle letture di `contrib`, i byte che ogni lettura trasferisce, in `double` o in `float`, e le copie fra CPU e GPU, molto più del numero di core o di SM. Diversamente da [3], che indica in CUDA la parallelizzazione più efficiente di PageRank, qui la risposta dipende dalla macchina.

4.5 Codice sorgente e guida all'uso

La repository è disponibile all'indirizzo

`https://github.com/EmanueleCecchia/pagerank-openmp-cuda`

Il `README.md` fa da guida all'uso e contiene le istruzioni per riscaricare i dataset, che non sono versionati per ragioni di dimensione.

Riferimenti bibliografici

[1] L. Page, S. Brin, R. Motwani, T. Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab, 1999.

[2] J. Leskovec, A. Krevl, *SNAP Datasets: Stanford Large Network Dataset Collection*, `https://snap.stanford.edu/data`, 2014.

[3] V. Visali, R. Manimegalai, S. Noor Mahammad, A. Sunitha Nandhini, *Performance Analysis of Parallelized PageRank Algorithm using OpenMP, MPI and CUDA*, ICSSEECC, pp. 44–49, IEEE, 2024.